

UiPathOrch マニュアル

目次

概要.....	7
最新のドキュメントにアクセスする	7
UiPathOrch の特徴.....	8
UiPathOrch がサポートする操作.....	9
フォルダーの操作	9
ライブラリパッケージの操作	9
パッケージの操作	9
プロセスの操作.....	9
個人用ワークスペースの操作	10
ライセンスの操作	10
セッションの操作	10
ジョブの操作	10
ジョブ記録の操作	10
マシンの操作	11
ロールの操作	11
ユーザーの操作.....	11
アセットの操作.....	11
キューの操作	12
トリガーの操作.....	12
API トリガーの操作.....	12
テストの操作	12
カレンダーの操作	12
ストレージバケットの操作.....	13
Webhook の操作	13
統計情報の操作.....	13
その他の操作	13
Platform Management の操作	13
Document Understanding の操作	14
UiPathOrch の操作.....	14
UiPathOrch のインストール.....	14

留意事項.....	14
インストールとセットアップの手順.....	15
UiPathOrch のバージョンアップ.....	19
権限の設定.....	19
スコープの設定.....	20
フォルダーにユーザーアカウントを追加.....	20
クイックスタート.....	21
PowerShell について.....	24
コマンドレットについて.....	24
コマンドレットの別名.....	24
コマンドレットのパラメーター.....	25
位置指定パラメーター.....	25
補完機能.....	25
実行したコマンドの履歴を呼び出すには.....	26
ワイルドカードについて.....	26
空白文字について.....	26
多くのコマンドレットで利用できるパラメーター.....	27
コマンドレットの出力結果を加工する.....	28
プロバイダーについて.....	30
マウント済みの PSDrive を確認する.....	30
ロケーションの操作.....	31
すべての UiPathOrch ドライブを一括して操作するには.....	33
UiPathOrch のコマンドレットについて.....	34
コマンドレット一覧.....	35
CSV ファイルへのエクスポート.....	39
CSV ファイルのインポート.....	40
エンティティをコピーする.....	41
コピー元フォルダーの指定.....	41
フォルダーをコピーする.....	42
個人用ワークスペースをコピーする.....	42
テナントを移行する.....	42
テナント移行の手順.....	43
テナント移行時の注意事項.....	46
コマンドレットリファレンス.....	46
フォルダーの操作.....	46
現在のロケーションを変更する.....	47

フォルダーを表示する	48
フォルダーをブラウザーで開く	49
フォルダーを作成する	49
フォルダーをコピーする	49
フォルダーを移動する	50
フォルダーをリネームする	51
フォルダーを削除する	51
フォルダーの使用状況を表示する	51
ライブラリパッケージの操作	52
ライブラリの最新バージョンを表示する	53
ライブラリのすべてのバージョンを表示する	53
ライブラリを削除する	54
ライブラリをアップロードする	54
ライブラリをダウンロードする	54
パッケージの操作	55
現在のフォルダーで利用可能なパッケージを表示する	56
現在のテナントのテナントフィールドにあるパッケージを表示する	56
現在のドライブのすべてのフィールドのパッケージを表示する	56
パッケージのすべてのバージョンを表示する	56
パッケージを削除する	57
パッケージをアップロードする	57
パッケージをダウンロードする	58
プロセスの操作	58
フォルダーに割り当てられたプロセスを表示する	59
プロセスをフォルダーから削除する	59
プロセスをブラウザーで編集する	60
フォルダーに割り当てられたプロセスのバージョンを更新する	60
個人用ワークスペースの操作	61
個人用ワークスペースを表示する	61
ライセンスの操作	61
Named User License を表示する	62
Runtime License を表示する	62
Runtime Licensee を有効にする	63
Runtime Licensee を無効にする	63
ジョブの操作	63
ジョブを表示する	64

ジョブをグループ化して数を数える	66
各ジョブの実行時間を求める	67
ジョブを開始する	67
ジョブを停止する	67
ジョブをブラウザで開く	68
ジョブ記録の操作	68
ビデオが添付されたジョブを取得する	69
スクリーンショットが添付されたジョブを取得する	69
スクリーンショットをファイルに保存する	69
スクリーンショットを削除する	70
ログの操作	70
ログを取得する	70
監査ログの操作	71
監査ログを取得する	72
ロボットの操作	72
テナントのすべてのロボットを表示する	72
指定した条件のロボットを表示する	73
ユーザーとグループの操作	73
テナントのユーザーを表示する	74
テナントのユーザーを削除する	74
現在のユーザーを確認する	75
現在のユーザーの Unattended Robot のパスワードを更新する	75
ロールの操作	75
ロールを表示する	76
ロールを削除する	76
テナント間でロールをコピーする	77
フォルダーユーザーの操作	77
フォルダーに割り当てられたユーザーを表示する	78
フォルダーからユーザーを削除する	79
フォルダーにユーザーを割り当てる	79
フォルダーに割り当て済みのユーザーから、ロールを削除する	80
マシンの操作	80
テナントのマシンを表示する	81
テナントのマシンを削除する	81
テナント間で、マシンをコピーする	81
フォルダーマシンの操作	81

フォルダーマシンを取得する	82
マシンをフォルダーに割り当てる	82
フォルダーマシンを削除する	83
アセットの操作.....	83
アセットとクレデンシャルアセットを表示する	84
アセットを追加・更新する	84
アセットを削除する	85
アセットを CSV ファイルにエクスポートする	86
アセットを CSV ファイルからインポートする	87
クレデンシャルアセットの操作	88
クレデンシャルアセットを表示する	88
クレデンシャルアセットを追加・更新する	88
クレデンシャルアセットを削除する	90
クレデンシャルアセットを CSV ファイルにエクスポートする	90
クレデンシャルアセットを CSV ファイルからインポートする	90
資格情報ストアの操作.....	91
資格情報ストアを取得する	91
資格情報ストアを削除する	92
キューの操作	92
キューを表示する.....	93
キューを削除する.....	93
キューアイテムを取得する	93
CSV ファイルをアップロードして、キューアイテムを一括登録する	94
トリガーの操作.....	94
トリガーを表示する	95
トリガーを削除する	95
トリガーを有効にする	96
トリガーを無効にする	96
テストの操作	96
テストケースを表示する.....	97
テストセットを表示する.....	97
テストセット実行を取得する	98
テストセットを実行する.....	98
テストセット実行を停止する	98
アラートの操作.....	99
アラートを表示する	99

その他の操作	99
--------------	----

概要

UiPath Orchestrator は、UiPath のロボティック・プロセス・オートメーション (RPA) プラットフォームの一部であり、自動化タスクの管理、監視、実行などのタスクを Web 画面から行えます。Web 画面での操作は便利ですが、Orchestrator 上の操作自体を自動化したり、スクリプトで操作したりすることは困難です。UiPathOrch は、この課題を解決するものであり、PowerShell のモジュールとして実装されています。

```
C:\Program Files\PowerShell\
PS Orch1:\> dir

Directory: Orch1:\

Id DisplayName Description FeedType ProvisionType
-----
765210 ff FolderHierarchy Automatic
709428 GSD-4466 受発注ファイル連携 FolderHierarchy Automatic
765172 hoge[fuga Processes Automatic
724985 hoge*fuga Processes Automatic
736424 hogehoge Processes Automatic
458334 new*name ほげ FolderHierarchy Automatic
724870 newHOGENAME Processes Automatic
291032 root Processes Automatic
738714 root2 Processes Automatic
725527 Shared Processes Automatic
492255 Shared5 xxx Processes Automatic
441637 uipath.settings.config Processes Automatic
738715 x1 Processes Automatic
765209 xxx Processes Automatic
765211 zzz FolderHierarchy Automatic
708734 クラシックだよ Processes Manual
678127 テストフォルダ Processes Automatic
300463 フィールドなしフォルダー Processes Automatic
707987 ほえほえ5 Processes Automatic
671751 完成 いえーい FolderHierarchy Automatic
706036 空白なし フィードつきですよー2 FolderHierarchy Automatic
221524 個人用ツールもろもろ FolderHierarchy Automatic
702134 新しいほえほえフォルダ Processes Automatic
702121 新フォルダ Processes Automatic
290195 総務部 Processes Automatic

PS Orch1:\> cd .\Shared\

ff root zzz 個人用ツールもろもろ
GSD-4466 受発注ファイル連携 root2 クラシックだよ
hoge[fuga Shared テストフォルダ
hoge*fuga Shared5 フィールドなしフォルダー
hogehoge uipath.settings.config ほえほえ5
new*name x1 完成
newHOGENAME xxx 空白なし

\Shared
```

最新のドキュメントにアクセスする

本ドキュメントは、UiPathOrch のインストールディレクトリに含まれています。現在、UiPathOrch は頻繁にアップデートされています。適宜、次のコマンドで最新版をインストールしてください。

```
PS> Update-Module UiPathOrch
```

本ドキュメントの最新版を参照するには、UiPathOrch のインストールディレクトリを開

いてください。UiPathOrch をインポートした後に、次のコマンドでインストールディレクトリに移動できます。

```
PS> Set-OrchLocation  
PS> ii.
```

なお、ii は、Invoke-Item の別名です。ピリオドはカレントフォルダーを示します。

UiPathOrch の特徴

- Orchestrator のテナントを PSDrive (PowerShell Drive) としてマウントし、Orchestrator のフォルダーを *dir*、*cd*、*mkdir* などの馴染みのあるコマンドで操作できるようにします。
- 複数のテナントをそれぞれ別のドライブとして、同時にマウントできます。各テナントは、別のアカウントで接続できます。
- Automation Cloud の Orchestrator と、オンプレミス Orchestrator の両方をサポートします。それぞれを別のドライブとして、同時に接続できます。
(オンプレミス Orchestrator 2022.10 で動作確認しました)
- コマンドラインから、多くの操作を簡単に実行できます。
- さまざまなエンティティを、カンマ区切りとワイルドカードで複数指定し、一括で操作できます。
- -Path パラメーターに複数のテナントとフォルダーを指定したり、-Recurse パラメーターを指定したりするなどにより、複数のテナントとフォルダーを一括で操作することもできます。
- スクリプトを作成したり、標準のコマンドレットと組み合わせたりすることで、より高度な自動化が行えます。
- パラメーター補完機能により、正しいパラメーター値を自動でコンソールに表示します。たとえば、フォルダー名やユーザー名、マシン名などを複数の候補値から選択して入力できます。また、現在のアセット値を表示し、これを編集して更新することもできます。
- 実行に際して、UiPath Assistant のインストールとライセンスは必要ありません。
- 非常に高速に動作します。可能な場合には、自動で複数のリクエストを同時に Orchestrator に送信し、戻ってきた各レスポンスをすぐにコンソールに出力します。
- いちど取得したエンティティの情報は、メモリ内にキャッシュします。同じ操作を繰り返した場合、キャッシュを使って即時に結果を返します。Orchestrator に対して、同じ API 呼び出しをすることはありません。(ログなど一部のエンティティについて

は、キャッシュせずにつど最新の状態を問い合わせます。) キャッシュのクリアを指示することもできます。

なお同様のツールとして、Orchestrator Manager があります。UiPathOrch は、Orchestrator Manager を完全に代替するものではないので、必要に応じて使い分けてください。

UiPathOrch がサポートする操作

フォルダーの操作

- モダンフォルダーの閲覧、削除、作成、移動、コピー
- クラシックフォルダーの閲覧、削除、コピー（モダンフォルダーに変換）
- フォルダーのサマリ情報の表示
- フォルダーをブラウザーで開く

ライブラリパッケージの操作

- テナントのライブラリフィールドにあるライブラリパッケージ名とバージョンの取得
- ライブラリパッケージの削除
- テナント間でライブラリパッケージを直接コピー
- ライブラリパッケージのアップロード
- ライブラリパッケージのダウンロード

パッケージの操作

- テナントのパッケージフィールドと、フィールドつきフォルダーにあるパッケージ名とバージョンの取得
- パッケージの削除
- パッケージを、同一のテナントの任意のフィールドへ直接コピー
- パッケージを、別のテナントの任意のフィールドへ直接コピー
- パッケージのアップロード
- パッケージのダウンロード

プロセスの操作

- プロセスの取得・追加・更新・削除・コピー

- フォルダーに割り当てられたプロセスを最新のバージョンに更新
- フォルダーに割り当てられたプロセスを固有のバージョンに更新
- フォルダーに割り当てられたプロセスを以前のバージョンにロールバック

個人用ワークスペースの操作

- 自分の個人用ワークスペースフォルダーを、dir コマンドで表示
- 個人用ワークスペースフォルダーの取得・削除
- 探索中の他人の個人用ワークスペースフォルダーを、dir コマンドで表示
- dir コマンドで表示中の個人用ワークスペースを、ほかのフォルダーと同様に操作（パッケージのアップロード/ダウンロードや、アセットの取得/更新など）

ライセンスの操作

- Named User License の取得
- Runtime License の取得
- Runtime License の有効化・無効化

セッションの操作

- ユーザーセッションの取得
- マシンセッションの取得

ジョブの操作

- ジョブの詳細をブラウザーで開く
- 各フォルダーにあるジョブの取得
- ジョブの開始・終了
- 各ジョブのログ取得

ジョブ記録の操作

- ジョブ記録（ビデオ）が添付されたジョブの取得
- ジョブ記録（スクリーンショット）が添付されたジョブの取得
- ジョブ記録（スクリーンショット）をファイルに保存
- ジョブ記録（スクリーンショット）の削除

マシンの操作

- マシンの取得・作成・削除
- テナント間で、テナントマシンをコピー
- フォルダーにマシンを割り当て・削除
- フォルダーに割り当てられたマシンを、同じもしくは別のテナントのフォルダーにコピー（割り当て）

ロールの操作

- ロールの取得・削除
- ロールに含まれる権限の確認
- テナント間で、ロールをコピー

ユーザーの操作

- テナントユーザーの取得・削除
- テナントユーザーにロールを追加
- テナントユーザーからロールを削除
- テナントユーザーを、テナント間でコピー
- ロボットアカウントを追加
- フォルダーに、ユーザーを割り当て・解除
- フォルダーに割り当てられたユーザーを、同じもしくは別のテナントのフォルダーにコピー
- フォルダーに割り当てられたユーザーに、ロールを割り当て・解除
- 現在の PowerShell セッションでテナントに接続中の、現在のユーザーを取得（非機密アプリで接続したドライブのみ）
- 現在のユーザーの Unattended Robot 用のパスワードを更新（非機密アプリで接続したドライブのみ）

アセットの操作

- アセットの取得・作成・更新・削除
- クレデンシャルアセットの取得・作成・更新・削除（資格情報ストアを指定可能）
- ユーザーごと・マシンごとのアセット/クレデンシャルアセットの作成・更新・削除
- アセットとクレデンシャルアセットを、同じもしくは別のテナントのフォルダーにコピー

- CSV ファイルのインポート・エクスポート
- 資格情報ストアの取得・削除

キューの操作

- キューの取得・追加・更新・削除・コピー
- キューアイテムの取得
- CSV ファイルをアップロードして、キューアイテムを一括登録

トリガーの操作

- フォルダー内にあるトリガーの取得・削除
- トリガーの有効化・無効化
- トリガーのコピー

API トリガーの操作

- フォルダー内にある API トリガーの取得・削除
- API トリガーの有効化・無効化
- API トリガーのコピー

テストの操作

- テストケースの取得・削除
- テストセットの取得・削除・実行
- テストセット実行の取得・停止
- テストセットスケジュールの取得・コピー・削除
- テストデータキューの取得・コピー・削除
- テストデータキューアイテムの取得

カレンダーの操作

- カレンダーの取得
- カレンダーの削除
- カレンダーを別のテナントにコピー

ストレージバケットの操作

- ストレージバケットの取得
- ストレージバケットの削除
- ストレージバケットのコピー

Webhook の操作

- Webhook の取得
- Webhook のコピー
- Webhook の削除
- Webhook の有効化・無効化

統計情報の操作

- ジョブ統計情報の取得
- ライセンス統計情報の取得

その他の操作

- ロボットの取得
- 監査ログの取得
- アラートの取得

Platform Management の操作

- ユーザーの取得
- ユーザーの削除
- ロボットアカウントの取得
- ロボットアカウントの作成と更新
- ロボットアカウントのコピー
- ロボットアカウントの削除
- グループの表示
- グループの削除
- グループにメンバーを追加
- グループからメンバーを削除
- 外部 API リソースの取得

- 外部アプリの取得

Document Understanding の操作

- プロジェクトの取得
- ドキュメントタイプの取得

UiPathOrch の操作

- UiPathOrch のメモリ内キャッシュをクリア
- UiPathOrch の設定ファイルをメモ帳で開く
- UiPathOrch のドライブを一覧表示
- UiPathOrch モジュールのインストールディレクトリに移動

UiPathOrch のインストール

留意事項

- UiPathOrch のインストールと実行には、PowerShell 7.x が必要です。
- UiPathOrch のインストールに管理者権限は不要です。現在のユーザーに対してインストールできます。
- UiPathOrch は PowerShell Gallery に登録されているため、PC がネットに接続されていれば簡単にインストールできます。
<https://www.powershellgallery.com/packages/UiPathOrch/>
- UiPathOrch は、Orchestrator への接続について、OAuth の機密アプリと非機密アプリの両方をサポートします。また、Orchestrator v20.10 以前に対しては、ユーザー名/パスワードで接続することもできます。
- OAuth 非機密アプリの設定が安全であるため、通常は非機密アプリを使うことをお勧めします。機密アプリによる接続は、無人の PC でスクリプトを実行する必要がある場合などで必要になることがあります。後続の手順では、非機密アプリの設定方法を紹介します。

接続方法	設定ファイルに アプリシークレットを記載	ユーザーがブラウ ザーでログイン	UiPathOrch に 付与される権限
OAuth 機密アプリ	必要 (外部に漏れないように注意)	不要	アプリスコープ

OAuth 非機密アプリ	不要（安全）	必要	ユーザースコープ
ユーザー名/ パスワード	パスワードを記載 (外部に漏れないように注意)	不要	N/A

インストールとセットアップの手順

1. PowerShell 7.x のインストール
2. UiPathOrch のインストール
3. OAuth 外部アプリケーションの登録
4. UiPathOrch のセットアップ

1. PowerShell 7.x のインストール

1-1. [Win+R]cmd[Enter] と入力し、コマンドプロンプトを起動。

1-2. 次のコマンドを入力し、PowerShell をインストール。

winget install --id Microsoft.Powershell --source winget

1-3. インストールが完了したら、コマンドプロンプトを閉じる。

2. UiPathOrch のインストール

2-1. [Win+R]pwsh[Enter] と入力し、PS コンソールを起動。

2-2. 次のコマンドを入力し、UiPathOrch モジュールをインストール。

Install-Module UiPathOrch

3. OAuth 外部アプリケーションの登録

本稿では、Automation Cloud での外部アプリケーション作成手順を説明します。オンプレミス版での外部アプリケーションの作成については、ユーザーガイドを参照してください。

■ Automation Cloud 外部アプリケーションを管理する

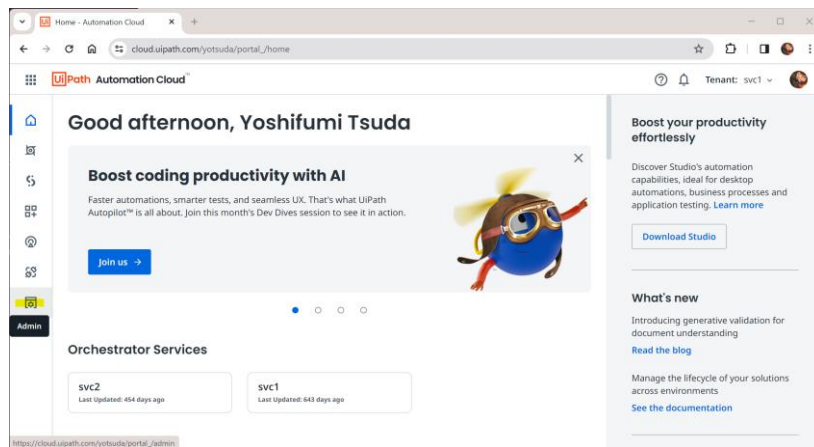
<https://docs.uipath.com/ja/automation-cloud/automation-cloud/latest/admin-guide/managing-external-applications>

■ オンプレ 外部アプリケーションを管理する

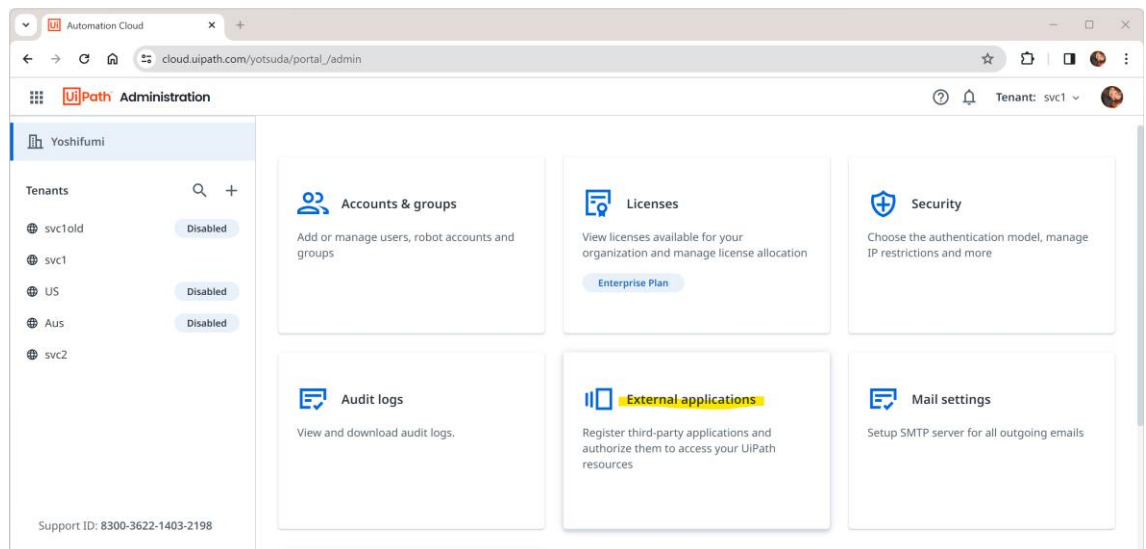
<https://docs.uipath.com/ja/orchestrator/standalone/2023.10/user-guide/managing-external-applications>

接続先 Orchestrator をブラウザで開き、OAuth 外部アプリケーションを作成します。
ここで払い出したアプリ ID は、あとで UiPathOrch の設定ファイルに記入します。

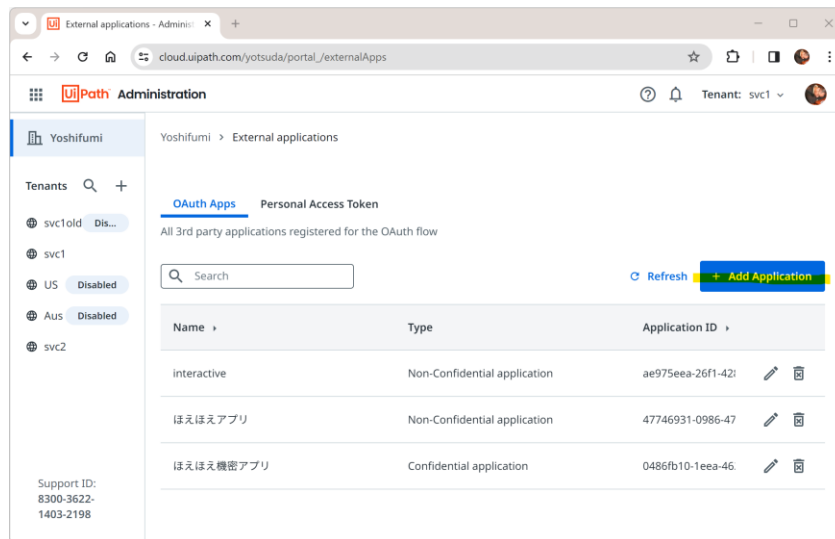
3-1. Orchestrator の管理画面を開く。



3-2. 外部アプリケーションを開く。



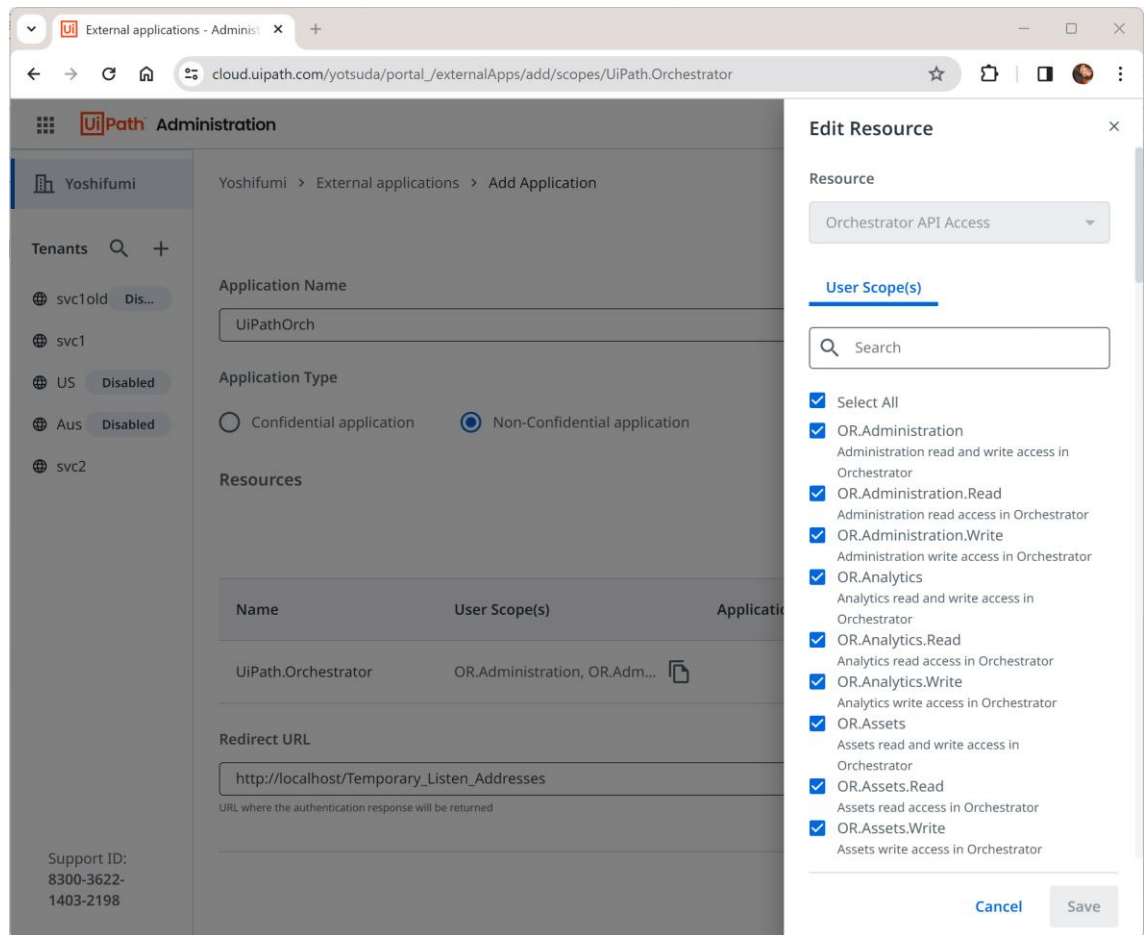
3-3. [アプリケーションを追加] をクリック。



- 3-4. [アプリケーション名]に任意の名前をつける。
- 3-5. [非機密アプリケーション]にチェックをつける。
- 3-6. [範囲を追加] をクリックし、リソース[Orchestrator API Access] の [ユーザーの範囲] をすべて選択して [保存] をクリック。
- 3-7. 同様に、[範囲を追加]をクリックし、リソース[Platform Management API Access]の[ユーザーの範囲]をすべて選択して[保存]をクリック。
- 3-8. [リダイレクト URL] に、次を入力。

`http://localhost/Temporary_Listen_Addresses`

- 3-9. [追加] をクリック。アプリ ID が表示されるので、これをクリップボードにコピー。



4. UiPathOrch のセットアップ

- 4-1. PS コンソールで次のコマンドを入力し、UiPathOrch モジュールをインポート。
 なお PSReadLine モジュールは、Ctrl+Space もしくは Tab を押下時の自動補完機能を動作させるために必要となります。

Import-Module UiPathOrch,PSReadLine

- 4-2. 初回のみ、自動で設定ファイルが作成され、メモ帳で開く。なお設定ファイルを手動で開くには、PS コンソールで *Edit-OrchConfig* を実行する。
- 4-3. Name が Orch1 となっている部分が非機密アプリの設定例なので、この部分の設定を書き換える。

- ✧ Name: PSDrive のドライブ名。Orch1 のままで問題ないが、マウントするテナント名と同じ名前にすることをお勧めします。
- ✧ Root: マウントする Orchestrator の URL を入力。
- ✧ AppId: 3.で払い出したアプリ ID を入力。

RedirectUrl、HttpListener、Scope を修正する必要はない。

- 4-3. 設定ファイルを保存して閉じる。
- 4-4. PS コンソールを閉じる。
- 4-5. [Win+R]pwsh[Enter] と入力し、PS コンソールを開き直す。
- 4-6. PS コンソールで次のコマンドを入力し、UiPathOrch モジュールをインポート。

Import-Module UiPathOrch

- 4-7. PS コンソールで次のコマンドを入力し、UiPathOrch の PSDrive がマウントされていることを確認。

Get-PSDrive

以上で、UiPathOrch が利用できる状態になりました。次回以降は、PS コンソールを起動して *Import-Module UiPathOrch* を実行するだけで、すぐに UiPathOrch が利用できます。

もし PS コンソール起動時に PSReadLine が無効化されている警告が表示された場合は、Tab もしくは Ctrl+Space による補完が動作しません。*Import-Module PSReadLine* を実行すると、補完が動作するようになります。

PS コンソールを起動時に自動で UiPathOrch を利用可能にするには、PS プロファイルに次の 1 行を記入して保存してください。

Import-Module UiPathOrch,PSReadLine

PS プロファイルをメモ帳で開くには、PS コンソールで *notepad \$profile* を実行します。

UiPathOrch のバージョンアップ

UiPathOrch は、適宜新しいバージョンがリリースされます。新しいバージョンの UiPathOrch をインストールするには、PS コンソールで次を実行します。

PS> Update-Module UiPathOrch

このコマンドを実行しても、既存の設定ファイルは削除されることなく、保持されます。

権限の設定

既定の設定では、安全のために利用できる操作が制限されています。必要な権限を付与してからご利用下さい。たとえば次のコマンドはテナント内のすべてのアセットを取得しますが、既定の状態では権限エラーのため結果を得られません。(OR.Assets.Read スコープが必要です)

PS Orch1:¥> Get-OrchAsset -Recurse

なお、必要ない権限を付与することは避けて下さい。たとえば、OR.Folders.Write 権限を付与すると、次のコマンドレットを実行できます。これは、テナント内のフォルダーをすべて、確認なしで即時に削除します。このようなエンティティを修正・削除する操作は元に戻せませんのでご注意ください。権限を付与しないでおけば、このようなコマンドを誤って実行しても、処理されることはありません。

```
PS Orch1:¥> rmdir *
```

スコープの設定

各コマンドレットを使うには、OAuth のスコープを追加する必要があります。Orchestrator の管理ページの外部アプリケーションにスコープを許可して、UiPathOrch 設定ファイルでスコープを要求してください。許可されていないスコープを要求すると、接続に失敗します。

- OAuth 非機密アプリに対しては、ユーザースコープを許可してください。
- OAuth 機密アプリに対しては、アプリスコープを許可してください。
- 各コマンドレットの実行に必要なスコープは、次のコマンドで確認できます。

```
PS> Get-Help <cmdlet name>
```

- スコープは、次の3つのいずれかの形式をしています。
 - OR.XXX
 - OR.XXX.Read
 - OR.XXX.Write
- OR.XXX は、OR.XXX.Read と OR.XXX.Write の両方を合わせたものです。OR.XXX を指定した場合には、OR.XXX.Read/OR.XXX.Write を指定する必要はありません。

フォルダーにユーザーアカウントを追加

各フォルダー内にあるエンティティを操作するには、そのフォルダーに自分自身のアカウントが適切なロールを伴って追加されていなければなりません。

本モジュールを使って、ルート直下のすべてのフォルダーにユーザーを追加するには、次のようにします。

```
PS Orch1:¥> Add-OrchFolderUser -Path * DirectoryUser <your name> 'Folder Administrator' -Verbose
```

上記のコマンドを実行するには、OR.Folders.Write 権限が必要です。また、管理者権限をもつユーザーが非機密アプリで接続する必要があります。上記では Folder Administrator ロールを付与していますが、必要に応じて適切なものを選択してください。

<your name> とロール名の入力時には、[Ctrl+Space] を押下すると自動補完され、簡単に

入力できます。また、コマンドレット名 (Add-OrchFolderUser) とパラメーター名 (-Path) の入力時にも自動補完が利用できます。パラメーター名を自動補完で入力するには、- (マイナス) を入力してから [Ctrl+Space] を押下します。

なお、現在非機密アプリで接続しているユーザー名は、次のコマンドレットで確認できます。このユーザーを各フォルダーに追加してください。このコマンドレットの実行には OR.Users.Read スコープが必要です。

```
PS Orch1:¥> Get-OrchCurrentUser
```

クイックスタート

UiPathOrch のセットアップが完了したら、PS コンソールを起動して *Import-Module UiPathOrch* を実行した後に、次のコマンドを実行してみてください。

- ***Get-PSDrive***

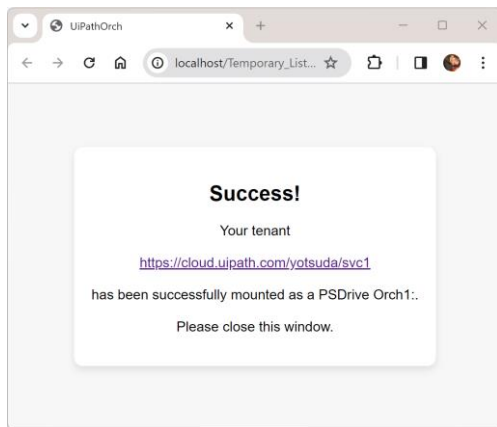
現在の PS セッションにマウントされている PSDrive が表示されます。

- ***cd Orch1:***

マウントされている Orch ドライブに移動します。設定ファイルでドライブ名を変更していれば、その名前を指定してください。(ドライブ名は、Get-PSDrive で確認できます) ドライブ名の最後には、コロンをつけてください。なお、ドライブ名とファイル名の大文字小文字は区別されません。

- ***dir***

ブラウザーが開き、Orchestrator へのログインが促されます。ログインすると、このテナントのフォルダーが PS コンソールに一覧表示されます。なお、非機密アプリで Orchestrator に接続した場合、UiPathOrch はあなたの権限で動作します。そのため、あなたがアクセスできないフォルダーは UiPathOrch を使ってもアクセスできないことに注意してください。



```

C:\Program Files\PowerShell\
PS C:\MyProj\OrchProvider\OrchProvider\bin\Debug\net7.0> cd orch1:
PS Orch1:\> dir

Directory: Orch1:\

Id DisplayName Description FeedType ProvisionType
-----
709428 GSD-4466 受発注ファイル連携 FolderHierarchy Automatic
724985 hoge*fuga Processes Automatic
736424 hogehoge Processes Automatic
458334 new*name ほげ FolderHierarchy Automatic
724870 newHOGENAME Processes Automatic
291032 root Processes Automatic
738714 root2 Processes Automatic
725527 Shared Processes Automatic
492255 Shared5 Processes Automatic
441637 uipath.settings.config xxx Processes Automatic
738715 x1 Processes Automatic
708734 クラシックだよ Processes Manual
678127 テストフォルダ Processes Automatic
300463 フィードなしフォルダー Processes Automatic
707987 ほえほえ5 Processes Automatic
671751 完成 いえーい FolderHierarchy Automatic
706036 空白なし フィードつきですよー2 FolderHierarchy Automatic
221524 個人用ツールもろもろ FolderHierarchy Automatic
702134 新しいほえほえフォルダ フォルダの説明 Processes Automatic
702121 新フォルダ Processes Automatic
290195 総務部 Processes Automatic

PS Orch1:\>

```

● *cd [Ctrl+Space]*

自動補完入力を試してください。移動先のフォルダーが一覧表示され、この中からフォルダーを選択して入力できます。

```

C:\Program Files\PowerShell\
PS Orch1:\> cd .\Shared\
ff root zzz 個人用ツールもろもろ
GSD-4466 受発注ファイル連携 root2 クラシックだよ
hoge[fuga Shared テストフォルダ
hoge*fuga Shared5 フィードなしフォルダー
hogehoge uipath.settings.config ほえほえ5
new*name x1 完成
newHOGENAME xxx 空白なし

\Shared

```

● *dir s**

sで始まるフォルダーだけが一覧表示されます。

- ***dir a[Ctrl+Space]***

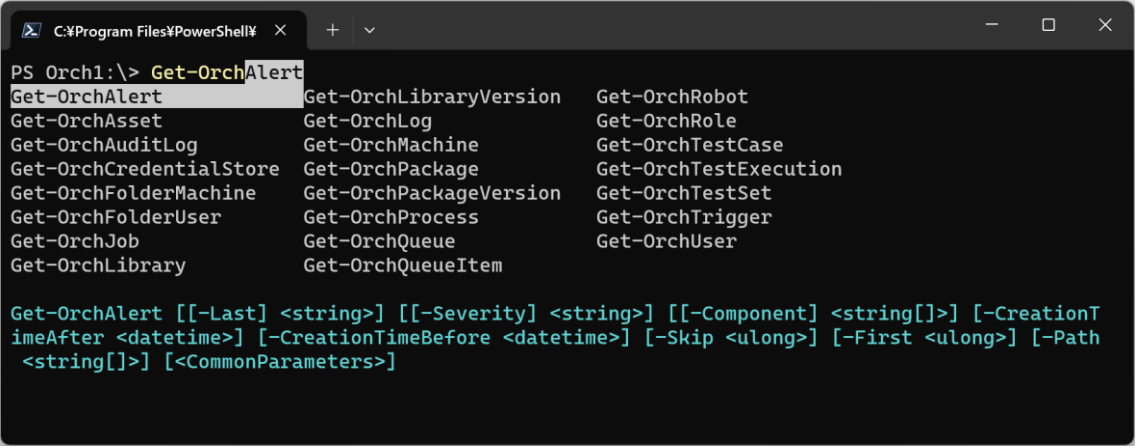
ワイルドカードのパターンを入力してから [Ctrl+Space] を押下すると、そのパターンに合致するフォルダーだけが一覧表示されます。上記の例では、a から始まるフォルダーだけが一覧表示されます。

- ***Get-Command -Module UiPathOrch***

UiPathOrch モジュールに含まれるコマンドをすべて表示します。

- ***Get-Orch[Ctrl+Space]***

コマンドレット名の補完入力も試してください。UiPathOrch のコマンドレット名において、名詞部分は必ず Orch で始まります。Get-Orch のほか、Add-Orch、Set-Orch、Remove-Orch などでも試してください。



```
C:\Program Files\PowerShell\> PS Orch1:\> Get-OrchAlert
Get-OrchAlert      Get-OrchLibraryVersion  Get-OrchRobot
Get-OrchAsset      Get-OrchLog             Get-OrchRole
Get-OrchAuditLog   Get-OrchMachine         Get-OrchTestCase
Get-OrchCredentialStore Get-OrchPackage         Get-OrchTestExecution
Get-OrchFolderMachine Get-OrchPackageVersion  Get-OrchTestSet
Get-OrchFolderUser  Get-OrchProcess         Get-OrchTrigger
Get-OrchJob         Get-OrchQueue           Get-OrchUser
Get-OrchLibrary     Get-OrchQueueItem

Get-OrchAlert [[-Last] <string>] [[-Severity] <string>] [[-Component] <string[]>] [-CreationTimeAfter <datetime>] [-CreationTimeBefore <datetime>] [-Skip <ulong>] [-First <ulong>] [-Path <string[]>] [<CommonParameters>]
```

- ***Get-OrchAsset -Recurse***

すべてのサブフォルダーのアセットを一覧表示します。

- ***Get-OrchAsset -Recurse -ExpandUserValue***

すべてのサブフォルダーのアセットの、ユーザー・マシンごとの値を一覧表示します。

- ***Get-OrchUser a****

名前が a で始まるユーザーをすべて表示します。a* の代わりに [Ctrl+Space] を押下することも試してください。

- ***Get-OrchUser a*,y****

名前が a で始まるユーザーと y で始まるユーザーをすべて表示します。カンマの後でも、

[Ctrl+Space]が有効に動作することも試してください。

ここまで、簡単に実行できるコマンドレットのサンプルを紹介しました。このほかのコマンドレットの詳細な使い方は、本稿のコマンドレットリファレンスを参照してください。

PowerShell について

UiPathOrch は、標準 PowerShell コマンドレットのマナーに沿うように設計されており、PowerShell に馴染みがあるユーザーはすぐに使い始めることができます。一方で、PowerShell を初めて使うユーザーには、分かりにくい部分があるかもしれません。本節では、UiPathOrch を活用する上で前提となる PowerShell に共通の操作について説明します。

コマンドレットについて

PowerShell コンソールで使えるコマンドを、コマンドレットといいます。コマンドレットの名前は、必ず **動詞-名詞** の形をしています。たとえば、よく使う標準コマンドレットには次のようなものがあります。

- *Set-Location*
- *Get-ChildItem*
- *Select-Item*
- *Format-List*
- *ConvertTo-Json*
- *Import-Csv*

このほか、大変多くのコマンドレットがあります。コマンドレット名とパラメーター名において、大文字と小文字は区別されません。

Hint: UiPathOrch モジュールにも多くのコマンドレットが含まれており、その名詞部分は必ず Orch で始まります。これらの使い方については後述します。

コマンドレットの別名

コマンドレットには別名が用意されているものがあります。たとえば、***Set-Location*** の別名は *cd*、*Get-ChildItem* の別名は *dir* です。これらは Windows のコマンドプロンプトや、Unix のシェルなどで同じ動作をするコマンドの名前なので、それらに馴染みのあるユーザーはすぐに PowerShell を使うことができます。別名の一覧は *Get-Alias* コマンドレットで確認できます。新しい別名を定義するには *Set-Alias* コマンドレットを使います。

コマンドレットのパラメーター

多くのコマンドレットが、パラメーターをもちます。パラメーターには、必ず名前があります。コマンドレット名と同じく、パラメーター名も大文字と小文字を区別しません。たとえば、*Set-Location* コマンドレットには *-Path* パラメーターがあります。パラメーター名の前には、*-* (ハイフン) がつきます。パラメーター名の直後に、その値を指定してください。たとえば、*-Path* パラメーターに値 *C:\Temp* を指定するには、次のようにします。

Set-Location -Path C:\Temp

なお、値を指定する必要がないパラメーターもあります。これをスイッチパラメーターといいます。たとえば、後述する *-Recurse*、*-WhatIf*、*-Confirm* などはスイッチパラメーターです。

位置指定パラメーター

パラメーターには、位置が定められているものがあります。これを位置指定パラメーターといいます。位置指定パラメーターに値を指定するときは、その名前を指定する必要はありません。たとえば前述のコマンドレット *Set-Location* の *-Path* は位置指定パラメーターであり、その位置はゼロ (コマンドレット名の直後) であると定められています。そのため、次のようにしても実行できます。

Set-Location C:\Temp

補完機能

コマンドレット名とパラメーター名を入力する際には、補完機能を利用できます。パラメーター値の入力時にも、補完機能を利用できるものがあります。補完機能は、[Tab]もしくは[Ctrl+Space]キー押下により、妥当な値の候補を表示します。UiPathOrchのコマンドレットは、すべてのパラメーター値で補完機能が利用できます。

なお、もし PowerShell コンソール起動直後に *PSReadLine* が無効化されている警告が表示された場合には、パラメーター補完機能は動作しません。これを動作するようにするには、*Import-Module PSReadLine* を実行してください。この恒久的な対処については、下記にあります。

<https://iwasi.hatenablog.jp/entry/2020/12/13/161312>

<https://serverfault.com/questions/1014754/cause-of-warning-powershell-detected-that-you-might-be-using-a-screen-reader-an>

UiPathOrchを使うときは、必ず *PSReadLine* をインポートしておくことをお勧めします。

実行したコマンドの履歴を呼び出すには

PS コンソールで、カーソルの上キーを押すと、これまでに実行したコマンドの履歴を呼び出せます。また、コマンドを入力中に、薄い字で入力予測が表示されることがあります。この薄い字を確定するには、カーソルの右キーを押します。

ワイルドカードについて

PowerShell コマンドレットに指定するパラメーターには、ワイルドカードを受け付けるものがあります。PowerShell で利用できるワイルドカードには次があります。

- * (アスタリスク)

任意の長さの任意のテキストに合致します。

- ? (クエスチョンマーク)

任意の 1 文字に合致します。

- [] (大かっこ)

大かっこで括った文字のうち、任意の一文字のいずれかに合致します。大かっこの中には複数の文字を指定できます。これは正規表現の大かっこに似ています。

上記の文字をワイルドカードとせず、字面通り解釈させるには、直前に ` (バッククォート) をつけてエスケープします。たとえば、名前にアスタリスクを含むアセット `my*asset` を表示するには次のようにします。

Get-OrchAsset my`*asset

なお、補完機能でアセット名などを入力したときは、ワイルドカード文字は自動でエスケープされます。

空白文字について

パラメーター値に空白文字を含むテキストを指定するには、テキストをシングルクォートで括ってください。シングルクォートで括られたテキスト内にシングルクォートを含めたいときは、そのシングルクォートをシングルクォートでエスケープしてください。たとえば、新規アセット `my asset` を作成し、その値に `that's it` を指定するには次のようにします。

Set-OrchAsset Text 'my asset' 'that's it'

多くのコマンドレットで利用できるパラメーター

操作対象のフォルダーを指定

- **-Path**

操作対象のディレクトリ（フォルダー）を指定します。UiPathOrch コマンドレットの **-Path** パラメーターは、ワイルドカードをサポートします。また、カンマ区切りで複数のフォルダーを指定することもできます。

- **-Recurse**

現在のフォルダーに加えて、子孫フォルダーを含むすべてのサブフォルダーに対しても操作することを指定するスイッチパラメーターです。

- **-Depth**

操作対象とするサブフォルダーの深さを指定します。たとえば 1 を指定すると、現在のフォルダーと、直接のサブフォルダーを対象として処理します。

Hint: UiPathOrch コマンドレットに **-Path/-Recurse/-Depth** を指定する必要があるときは、なるべくほかのパラメーターよりも先(コマンドレット名の直後)に指定してください。これにより、後続のパラメーターの補完機能が適切に動作するようになります。

- **-Name**

操作対象とするエンティティを指定します。UiPathOrch のコマンドレットの **-Name** パラメーターは、ワイルドカードをサポートします。また、カンマ区切りで複数の名前を指定することもできます。

Hint: ほとんどの UiPathOrch コマンドレットで、**-Name** パラメーターは位置がゼロの位置指定パラメーターとなっているため、このパラメーター名の入力は省略できます。

リスク軽減パラメーター

- **-WhatIf**

エンティティを作成・更新・削除するコマンドレットで利用できるスイッチパラメーターです。実際に作成・更新・削除をすることなく、どのような操作を行うかを表示します。たとえばパラメーター値をワイルドカードで指定したとき、このワイルドカードがどのように展開されるかを確認できます。UiPathOrch のコマンドレットを利用する上で、非常に重要で便利なパラメーターです。

- **-Confirm**

このスイッチパラメーターは、各エンティティを操作する直前に確認メッセージを表示し、

本当に操作を実行するかをユーザーに問い合わせます。上述の-WhatIf パラメーターを利用できるコマンドレットは、必ず-Confirm パラメーターも利用できます。

共通パラメーター

すべてのコマンドレットで利用できるパラメーターです。このほかにもいくつかの共通パラメーターがありますが、本稿では一部のみ紹介します。

https://learn.microsoft.com/ja-jp/powershell/module/microsoft.powershell.core/about/about_commonparameters

- **-Verbose**

通常、コマンドレットを実行しても、どのような処理を試行したかについてはコンソールに表示されません。これを（-WhatIf と同じように）コンソールに表示することを指示するには、-Verbose パラメーターを指定します。このパラメーターの別名は-vb です。

- **-ErrorAction**

処理対象を（ワイルドカードとカンマ区切りで）複数指定したとき、途中でエラーが発生した場合に後続の処理をどうするか指定するパラメーターです。既定値は Continue で、エラーが発生してもメッセージを表示した上で後続の処理を継続します。ここに Stop を指定すると、エラーが発生した場合にはメッセージを表示して停止します。後続の処理はキャンセルされます。このパラメーターの別名は-ea です。

出力行数を制限

- **-Skip**

指定した行数を、先頭からスキップして表示します。たとえば、-Skip 5 と指定すると、先頭の 5 行をスキップします。出力される行数が多いときに便利です。

- **-First**

指定した行数だけを表示します。上述の-Skip と組み合わせて利用できます。

コマンドレットの出力結果を加工する

各コマンドレットは、ひとつのエントリを 1 行で出力します。各エントリには複数のデータがフィールドとして含まれていますが、既定でコンソールに出力されるのはビューで定義されたフィールドのみです。出力結果を加工するには、次のような方法があります。

- ***Get-OrchAsset / Format-List***

既定では、コマンドレットが出力する各エントリは Table 形式（*Format-Table* コマンドレット）でコンソールに表示されます。しかし、Table 形式では出力されるフィールドが制限されており、エントリに含まれるすべてのフィールドは表示され

ません。List 形式で出力すると、多くのフィールドの内容を確認できます。

- ***Get-OrchAsset / select Name, Value***

指定のフィールドだけを Table 形式で出力するには、Select-Object コマンドレットを使います。select は Select-Object の別名です。Select に指定するフィールド名は補完入力できます。

- ***Get-OrchAsset / ConvertTo-Json***

コマンドレットが出力するエンティティのフィールドには、ネストしたものがあります。たとえばアセットのエンティティには、ユーザー・マシンごとのアセット値がネストしたフィールドに含まれています。これらの値は、List 形式のビューや select でも内容を確認できません。このようなエンティティは Convert-Json コマンドレットで Json 形式に変換すると、ネストしたフィールド値をすべて確認できます。

- ***Get-OrchAsset -ExpandUserValue***

コマンドレットによっては、ネストしたフィールドの内容を展開してコンソールに出力するためのスイッチパラメーターが用意されています。*Get-OrchAsset* の *-ExpandUserValue* は、ユーザー・マシンごとの値をすべて展開して表示します。同様の用途のため、*Get-OrchRole* には *-ExpandPermission* パラメーターが、*Get-OrchAuditLog* には *-ExpandEntity* パラメーターが用意されています。

- ***Get-OrchAsset / ? <列名> -eq <値>***

条件に合致するエンティティだけをフィルターして出力するには、? にパイプ出力します。たとえば、値が False であるアセットをすべて取得するには *Get-OrchAsset / ? {\$_.Value -eq \$false}* とします。? は、*Where-Object* の別名です。-eq は equal の意味です。なお多くの UiPathOrch コマンドレットにはサーバー側でエンティティをフィルターするパラメーターがあることに留意してください。一般に、サーバー側で処理した方がネットワークに負荷をかけません。一方で、クライアント側ではより柔軟な条件でフィルターできます。両方を組み合わせて実行することもできます。

<https://learn.microsoft.com/ja-jp/powershell/scripting/samples/removing-objects-from-the-pipeline--where-object->

https://learn.microsoft.com/ja-jp/powershell/module/microsoft.powershell.core/about/about_simplified_syntax

- ***Get-OrchAsset > c:output.txt***

> は、ファイルへのリダイレクト出力を指示します。上記は、c:ドライブのカレントロケーションに output.txt を作成します。なお c:ドライブのカレントロケーションに移動するには、cd c:とします。c:ドライブのルートフォルダーに output.txt を作成するには、> c:¥output.txt とします。

- ***Get-OrchAsset -ExportCsv c:***

- ***Get-OrchAsset -ExportCredentialCsv c:***

Get-OrchAsset コマンドレットには、アセットとクレデンシャルアセットを CSV ファイルにエクスポートすることを指示するパラメーターがあります。これらの CSV ファイルは、*Set-OrchAsset* と *Set-OrchCredentialAsset* でインポートできます。この方法は後述します。

- ***Get-OrchLog <JobId> / Export-Csv C:¥<output-path>¥log.csv -Encoding utf8BOM***
-ExportCsv パラメーターをサポートしないコマンドレットにおいては、その出力を Export-Csv コマンドレットにパイプ出力することで、CSV ファイルを出力できます。
- ***Get-OrchJob / select ReleaseName,JobPriority,StartTime / Export-Csv C:¥<output-path>¥log.csv -Encoding utf8BOM***
Export-Csv は、前述の select と併用できます。出力する CSV ファイルの項目を柔軟に構成できます。

プロバイダーについて

PowerShell のプロバイダーは、ツリー構造のデータをドライブとしてマウントし、このデータを *cd* や *mkdir* などのコマンドで操作できるようにします。標準のプロバイダーには、たとえば次のものがあります。

- **FileSystem**
ハードディスクなどの外部記憶装置に含まれるディレクトリとファイルを操作します。
- **Registry**
Windows のレジストリデータベースを操作します。レジストリもツリー構造をしているため、PowerShell で快適に操作できます。
- **Variable**
Windows の環境変数を操作します。環境変数はツリー構造をしている訳ではありませんが、PowerShell コンソール上で統一されたコマンド群により環境変数を操作できます。

Hint: UiPathOrch も PowerShell のプロバイダーです。ファイルシステムやレジストリと同じ操作で、Orchestrator のモダンフォルダーを扱うことができます。

マウント済みの PSDrive を確認する

PSDrive (PowerShell Drive) とは、プロバイダーによりマウントされたドライブです。*Get-PSDrive* コマンドレットにより、現在マウントされている PSDrive の一覧を確認できます。

```
C:\Program Files\PowerShell\ > Get-PSDrive

Name      Used (GB)  Free (GB) Provider      Root      CurrentLocation
-----
Alias      251.02     223.69   Alias         C:\        Users\yoshifumi.tsuda
C          251.02     223.69   FileSystem    C:\        \
Cert                               Certificate   \
Env                               Environment
Function                               Function
HKCU                               Registry     HKEY_CURRENT_USER
HKLM                               Registry     HKEY_LOCAL_MACHINE
Orch1      251.02     223.69   UiPathOrch    https://cloud.uipath.com/yotsuda/s...
Orch2                               UiPathOrch    https://cloud.uipath.com/yotsuda/s...
Orch3                               UiPathOrch    https://oc2210
Orch4                               UiPathOrch    https://oc2210
Temp      251.02     223.69   FileSystem    C:\Users\yoshifumi.tsuda\AppData\L...
Variable                               Variable
WSMan                               WSMAN

PS C:\Users\yoshifumi.tsuda>
```

Hint: 通常、PSDrive は *New-PSDrive* コマンドレットでマウントしますが、UiPathOrch の PSDrive をマウントするには設定ファイルを記述する必要があります。この手順は後述します。

Hint: コンソールをクリアするには、*cls* を実行します。cls は clear screen の略です。

ロケーションの操作

PowerShell コマンドレットは、既定ではカレントロケーション（現在の場所）を処理対象にします。現在のドライブの現在の場所は、PowerShell コンソールのプロンプトに表示されます。

たとえば、次の PS コンソールのカレントロケーションは C:\Users です。

```
C:\Program Files\Power >

PS C:\Users>
```

カレントロケーションは、ドライブごとに保持されます。コマンドレットで、カレントロケーション以外のフォルダーを操作したいときは、その場所を -Path パラメーターで指定してください。なお -Path パラメーターにドライブ名だけを指定すると、そのドライブのカレントロケーションを指定したことになります。

<https://learn.microsoft.com/ja-jp/powershell/scripting/samples/managing-current-location>

Path の例	説明
.	カレントドライブのカレントフォルダー
..	カレントドライブのカレントフォルダーの親フォルダー
Orch1:	Orch1:ドライブのカレントフォルダー
Orch1:¥	Orch1:ドライブのルートフォルダー
Orch2:	Orch2:ドライブのカレントフォルダー
Orch2:¥	Orch2:ドライブのルートフォルダー
Orch2:..	Orch2:ドライブのカレントフォルダーの親フォルダー
Orch2:..¥..	Orch2:ドライブのカレントフォルダーの親フォルダーの親フォルダー
Orch1:sub	Orch1:ドライブのカレントフォルダー直下にある sub フォルダー
Orch1:¥sub	Orch1:ドライブのルートフォルダー直下にある sub フォルダー

例

● *Get-PSDrive*

すべてのドライブのカレントロケーションを確認します。

● *Get-Location*

現在のドライブのカレントロケーションを確認します。この別名は *pwd* です。これは *print working directory* の略です。

● *Set-Location c:*

Set-Location コマンドレットにドライブ名を指定すると、各ドライブのカレントロケーションに移動します。この別名は *cd* です。これは *change directory* の略です。

● *cd c:¥*

指定したドライブのルートフォルダーに移動します。ルートフォルダーは ¥ 記号で指定します。

● *cd ..*

現在のロケーションの親フォルダーに移動します。親フォルダーは .. で指定します。なお、現在のフォルダーは . で指定します。

● *Push-Location <移動先フォルダー>*

現在のロケーションを記録してから、別のフォルダーに移動します。このあと *Pop-Location* を実行すると、元の（記録した）ロケーションに戻れます。それぞれの別名は、*pushd* と *popd* です。

- ***Get-ChildItem***

現在のロケーションにあるアイテム（サブフォルダーとファイル）を表示します。この別名は *dir* です。これは *directory* の略です。

- ***dir -Recurse***

現在のロケーションと、そのサブフォルダーにあるアイテムをすべて表示します。

- ***dir -Depth 1***

現在のロケーションと、その直接のサブフォルダーにあるアイテムを表示します。

- ***ii.***

現在のロケーションを、ファイルエクスプローラーで開きます。

Hint: 一般に、プロバイダーで扱える項目 (Item) にはコンテナ (フォルダー) と非コンテナ (ファイル) があり、*Get-ChildItem* (*dir*) コマンドはその両方を表示します。しかし UiPathOrch プロバイダーにおいては、*Get-ChildItem* はコンテナ (フォルダー) だけを表示します。非コンテナの項目 (ユーザー、マシン、アセット、トリガーなど) を表示するには、それぞれのエンティティに対応したコマンドレットを使います。たとえば、*Get-OrchUser*、*Get-OrchMachine*、*Get-OrchAsset*、*Get-OrchTrigger* などのコマンドレットがあります。

すべての UiPathOrch ドライブを一括して操作するには

複数の UiPathOrch ドライブをマウントして、それらをまとめて操作したいことがあります。たとえば、マウントした UiPathOrch ドライブのすべてのフォルダーから、プロセスを取得するには次のようにします。

Get-OrchProcess -Recurse -Path Orch1:¥,Orch2:¥,Orch3:¥

しかし、マウントした UiPathOrch ドライブの数が多いとき、上記のようにドライブ名を手動で列挙するのは大変です。これは、*Get-PSDrive* コマンドレットで列挙することができます。次のようにします。

```
# UiPathOrch プロバイダーのドライブ名の一覧を取得し、変数に保存します
$drivePaths = (Get-PSDrive -PSProvider UiPathOrch) | ForEach-Object { "$($_.Name):¥" }

# 各ドライブからプロセスを取得し、CSV ファイルに保存します
Get-OrchProcess -Recurse -Path $drivePaths | Export-Csv -Path c:¥tmp¥all-processes.csv
```

マウントされているすべてのテナントのプロセスを、上記の方法で取得するサンプルスクリプトが、UiPathOrch インストールフォルダにあります。

Examples¥Get-ProcessesFromAllOrchDrives.ps1

UiPathOrch のコマンドレットについて

- 本来、Orchestrator のモダンフォルダーにおいて、ルートフォルダーは存在しません。しかし UiPathOrch のプロバイダーにおいては、ルートフォルダーを利用できます。これは、テナントエンティティを操作するときや、-Recurse パラメーターやワイルドカードを使って複数のフォルダーを操作対象とするときに便利です。
- 各コマンドレットの-Path、-Recurse、-Depth などのパラメーターで、操作対象のフォルダーを指定してください。これらを指定しない場合、操作対象はカレントロケーションとなります。
- -Path、-Recurse、-Depth などのパラメーターを指定するときは、コマンドレット名の直後に指定してください。これにより、後続のパラメーター値の補完機能が適切に動作するようになります。
- Orchestrator のフォルダーの区切り文字は / (スラッシュ) ですが、UiPathOrch は区切り文字を ¥ (バックスラッシュ) で表示します。ただし、コマンドレットに-Path パラメーターでフォルダーを指定するときは / も区切り文字として使えます。

Hint: C:ドライブ (FileSystem プロバイダ) のフォルダーの区切り文字も¥で表示されますが、コマンドレットで指定するときは¥もしくは/の両方が使えます。

- 補完機能は、コマンドレット名、パラメーター名、パラメーター値について動作します。
 - コマンドレット名を補完するには、get-orch[Ctrl+Space]と入力します。Get のほか、set、add、remove などの動詞で始まるコマンドレットがあります。
 - パラメーター名を補完するには、コマンドレット名と空白を入力後に、-[Ctrl+Space]を入力します。
 - パラメーター値を補完するには、コマンドレット名、空白、パラメーター名、空白を入力後に [Ctrl+Space] を入力します。位置指定パラメーターの値を入力する場合は、パラメーター名は省略できます。
 - パラメーター値を補完するとき、ワイルドカードを含むテキストを入力してから [Ctrl+Space]を押下すると、表示される候補リストを絞り込むことができます。

コマンドレット一覧

分類	コマンドレット名 (別名)	概要
フォルダー	<i>Get-Location (pwd)</i>	現在のフォルダーを表示
	<i>Set-Location (cd)</i>	現在のフォルダーを変更
	<i>Get-ChildItem (dir)</i>	サブフォルダーを表示
	<i>New-Item (mkdir)</i>	フォルダーを作成
	<i>Move-Item (mv)</i>	フォルダーを移動
	<i>Copy-Item (copy)</i>	フォルダーをコピー
	<i>Rename-Item (ren)</i>	フォルダー名を変更
	<i>Remove-Item (rm)</i>	フォルダーを削除
	<i>Invoke-Item (ii)</i>	フォルダーをブラウザで開く
	<i>Get-OrchFolderUsage</i>	フォルダーの使用状況を表示
ライブラリ	<i>Get-OrchLibrary</i>	最新のライブラリパッケージを取得
	<i>Get-OrchLibraryVersion</i>	ライブラリパッケージのバージョンを取得
	<i>Copy-OrchLibrary</i>	ライブラリパッケージをコピー
	<i>Remove-OrchLibrary</i>	ライブラリパッケージを削除
	<i>Import-OrchLibrary</i>	ライブラリパッケージをアップロード
	<i>Export-OrchLibrary</i>	ライブラリパッケージをダウンロード
パッケージ	<i>Get-OrchPackage</i>	最新のパッケージを取得
	<i>Get-OrchPackageVersion</i>	パッケージのバージョンを取得
	<i>Copy-OrchPackage</i>	パッケージをコピー
	<i>Remove-OrchPackage</i>	パッケージを削除
	<i>Import-OrchPackage</i>	パッケージをアップロード
	<i>Export-OrchPackage</i>	パッケージをダウンロード
個人用ワークスペース	<i>Get-OrchPersonalWorkspace</i>	個人用ワークスペースを取得
	<i>Remove-OrchPersonalWorkspace</i>	個人用ワークスペースを削除
ライセンス	<i>Get-OrchLicenseNamedUser</i>	Named User License を取得
	<i>Get-OrchLicenseRuntime</i>	Runtime License を取得
	<i>Enable-OrchLicenseRuntime</i>	Runtime License を有効化
	<i>Disable-OrchLicenseRuntime</i>	Runtime License を無効化
統計情報	<i>Get-OrchJobStats</i>	ジョブの統計情報を取得
	<i>Get-OrchLicenseStats</i>	ライセンスの統計情報を取得
ジョブ	<i>Get-OrchJob</i>	ジョブを取得
	<i>Start-OrchJob</i>	ジョブを開始

	<i>Stop-OrchJob</i>	ジョブを終了
	<i>Open-OrchJob</i>	ジョブの詳細をブラウザで開く
ジョブ記録	<i>Get-OrchJobVideo</i>	ジョブ記録(ビデオ)が添付されたジョブを取得
	<i>Get-OrchJobMedia</i>	ジョブ記録(スクリーンショット)が添付されたジョブを取得
	<i>Export-OrchJobMedia</i>	ジョブ記録(スクリーンショット)をファイルに保存
	<i>Remove-OrchJobMedia</i>	ジョブからジョブ記録(スクリーンショット)を削除
ログ	<i>Get-OrchLog</i>	ジョブの実行ログを取得
	<i>Get-OrchAuditLog</i>	監査ログを取得
プロセス	<i>Get-OrchProcess</i>	フォルダーに割り当てられたプロセスを取得
	<i>New-OrchProcess</i>	フォルダーにプロセスを作成
	<i>Update-OrchProcess</i>	フォルダー内にあるプロセスを更新
	<i>Remove-OrchProcess</i>	フォルダーからプロセスを削除
	<i>Edit-OrchProcess</i>	プロセスをブラウザで編集
	<i>Update-OrchProcessVersion</i>	フォルダーに割り当てられたプロセスのバージョンを更新
ロボット	<i>Reset-OrchProcessVersion</i>	フォルダーに割り当てられたプロセスのバージョンをロールバック
	<i>Get-OrchRobot</i>	ロボットを取得
カレンダー	<i>Add-OrchRobotAccount</i>	ロボットアカウントを追加
	<i>Get-OrchCalendar</i>	カレンダーを取得
	<i>Add-OrchCalendarDate</i>	カレンダーに日付を追加
	<i>Remove-OrchCalendarDate</i>	カレンダーから日付を削除
	<i>Copy-OrchCalendar</i>	カレンダーを別のテナントにコピー
ユーザー	<i>Remove-OrchCalendar</i>	カレンダーを削除
	<i>Get-OrchUser</i>	ユーザー・グループを取得
	<i>Add-OrchUser</i>	ユーザー・グループを追加
	<i>Copy-OrchUser</i>	ユーザーをテナント間でコピー
	<i>Remove-OrchUser</i>	ユーザーを削除
	<i>Add-OrchRoleToUser</i>	ユーザーにロールを追加
	<i>Remove-OrchRoleFromUser</i>	ユーザーからロールを削除
	<i>Get-OrchCurrentUser</i>	この PowerShell セッションでテナントに接続中の現在のユーザーを取得

	<i>Update-OrchCurrentUserURPassword</i>	現在のユーザーの Unattended Robot 用パスワードを更新
ロール	<i>Get-OrchRole</i>	ロールを取得
	<i>Remove-OrchRole</i>	ロールを削除
	<i>Copy-OrchRole</i>	テナント間でロールをコピー
フォルダー ユーザー	<i>Get-OrchFolderUser</i>	フォルダーに割り当てられたユーザーを取得
	<i>Add-OrchFolderUser</i>	フォルダーにユーザーを割り当て
	<i>Remove-OrchFolderUser</i>	フォルダーからユーザーを削除
	<i>Copy-OrchFolderUser</i>	フォルダーに割り当てられたユーザーを、別のフォルダーにコピー（割り当て）
	<i>Add-OrchRoleToFolderUser</i>	フォルダーに割り当てられたユーザーにロールを追加
	<i>Remove-OrchRoleFromFolderUser</i>	フォルダーに割り当てられたユーザーからロールを削除
マシン	<i>Get-OrchMachine</i>	マシンを取得
	<i>New-OrchMachine</i>	マシンを作成
	<i>Copy-OrchMachine</i>	テナント間でマシンをコピー
	<i>Remove-OrchMachine</i>	マシンを削除
フォルダー マシン	<i>Get-OrchFolderMachine</i>	フォルダーに割り当てられたマシンを取得
	<i>Add-OrchFolderMachine</i>	フォルダーにマシンを割り当て
	<i>Remove-OrchFolderMachine</i>	フォルダーからマシンを削除
アセット	<i>Get-OrchAsset</i>	アセット・クレデンシャルアセットを取得
	<i>Set-OrchAsset</i>	アセットを追加・更新・削除
	<i>Set-OrchCredentialAsset</i>	クレデンシャルアセットを追加・更新・削除
	<i>Copy-OrchAsset</i>	アセットとクレデンシャルアセットを別のフォルダーにコピー
	<i>Remove-OrchAsset</i>	アセットを削除
資格情報 ストア	<i>Get-OrchCredentialStore</i>	資格情報ストアを取得
	<i>Remove-OrchCredentialStore</i>	資格情報ストアを削除
Webhook	<i>Get-OrchWebhook</i>	Webhook を取得
	<i>Copy-OrchWebhook</i>	Webhook をコピー
	<i>Remove-OrchWebhook</i>	Webhook を削除
	<i>Enable-OrchWebhook</i>	Webhook を有効化
	<i>Disable-OrchWebhook</i>	Webhook を無効化
キュー	<i>Get-OrchQueue</i>	キューを取得
	<i>New-OrchQueue</i>	キューを作成

	<i>Update-OrchQueue</i>	キューを更新
	<i>Copy-OrchQueue</i>	キューをコピー
	<i>Remove-OrchQueue</i>	キューを削除
	<i>Get-OrchQueueItem</i>	キューアイテムを取得
	<i>Import-OrchQueueItem</i>	CSV ファイルをアップロードして、キューアイテムを一括登録
トリガー	<i>Get-OrchTrigger</i>	トリガーを取得
	<i>Remove-OrchTrigger</i>	トリガーを削除
	<i>Copy-OrchTrigger</i>	トリガーをコピー
	<i>Enable-OrchTrigger</i>	トリガーを有効化
	<i>Disable-OrchTrigger</i>	トリガーを無効化
API トリガー	<i>Get-OrchApiTrigger</i>	API トリガーを取得
	<i>Remove-OrchApiTrigger</i>	API トリガーを削除
	<i>Copy-OrchApiTrigger</i>	API トリガーをコピー
	<i>Enable-OrchApiTrigger</i>	API トリガーを有効化
	<i>Disable-OrchApiTrigger</i>	API トリガーを無効化
ストレージバケット	<i>Get-OrchBucket</i>	ストレージバケットを取得
	<i>Get-OrchBucketItem</i>	ストレージバケット内のアイテムを取得
	<i>Remove-OrchBucket</i>	ストレージバケットを削除
	<i>Copy-OrchBucket</i>	ストレージバケットをコピー
テストケース	<i>Get-OrchTestCase</i>	テストケースを取得
	<i>Remove-OrchTestCase</i>	テストケースを削除
テストセット	<i>Get-OrchTestSet</i>	テストセットを取得
	<i>Remove-OrchTestSet</i>	テストセットを削除
	<i>Start-OrchTestSet</i>	テストセットを実行
テスト実行	<i>Get-OrchTestExecution</i>	テストセット実行を取得
	<i>Stop-OrchTestExecutoin</i>	テストセット実行を停止
テストスケジュール	<i>Get-OrchTestSetSchedule</i>	テストスケジュールを取得
	<i>Copy-OrchTestSetSchedule</i>	テストスケジュールをコピー
	<i>Remove-OrchTestSetSchedule</i>	テストスケジュールを削除
テストデータキュー	<i>Get-OrchTestDataQueue</i>	テストデータキューを取得
	<i>Copy-OrchTestDataQueue</i>	テストデータキューをコピー
	<i>Remove-OrchTestDataQueue</i>	テストデータキューを削除
テストデータキューアイテム	<i>Get-OrchTestDataQueueItem</i>	テストデータキューアイテムを取得

アラート	<i>Get-OrchAlert</i>	アラートを取得
セッション	<i>Get-OrchUserSession</i>	ユーザーセッションを取得
	<i>Get-OrchMachineSession</i>	マシンセッションを取得
カレンダー	<i>Get-OrchCalendar</i>	カレンダーを取得
	<i>Remove-OrchCalendar</i>	カレンダーを削除
	<i>Copy-OrchCalendar</i>	カレンダーをコピー
プラットフォーム管理 の操作	<i>Get-PmUser</i>	ユーザーを取得
	<i>Remove-PmUser</i>	ユーザーを削除
	<i>Get-PmRobotAccount</i>	ロボットアカウントを取得
	<i>Set-PmRobotAccount</i>	ロボットアカウントを作成・更新
	<i>Copy-PmRobotAccount</i>	ロボットアカウントを別のテナントにコピー
	<i>Remove-PmRobotAccount</i>	ロボットアカウントを削除
	<i>Get-PmGroup</i>	グループを取得
	<i>New-PmGroup</i>	グループを作成
	<i>Remove-PmGroup</i>	グループを削除
	<i>Add-PmGroupMember</i>	グループにメンバーを追加
	<i>Remove-PmGroupMember</i>	グループからメンバーを削除
	<i>Get-PmExternalApiResource</i>	外部 API リソースを取得
	<i>Get-PmExternalApplication</i>	外部アプリを取得
Document Understanding の操作	<i>Get-DuProject</i>	プロジェクトを取得
	<i>Get-DuDocumentType</i>	ドキュメントタイプを取得
その他	<i>Clear-OrchCache</i>	UiPathOrch のメモリ内キャッシュをクリア
	<i>Edit-OrchConfig</i>	UiPathOrch の設定ファイルを開く
	<i>Get-OrchPSDrive</i>	UiPathOrch ドライブを一覧表示
	<i>Set-OrchLocation</i>	UiPathOrch のインストールディレクトリに移動

CSV ファイルへのエクスポート

「コマンドレットの出力結果を加工する」でご紹介したように、すべての *Get-OrchXxx* コマンドレットの出力は次のようにして好きな書式でエクスポートできます。列名は [Ctrl+Space] で補完入力できます。ファイルパスについては、ドライブ名(c:)は手動で入力する必要がありますが、c:の後に [Ctrl+Space] を押下するとフォルダーを補完入力できま

す。

```
PS> Get-OrchXxx | select <列名#1>,<列名#2> | Export-Csv <ファイルパス>
```

また、次のコマンドレットは、インポート可能な書式で簡単にエクスポートできるように -ExportCsv パラメーターが用意されています。このとき、-CsvEncode パラメーターで CSV ファイルのエンコーディングを指定できます。

-ExportCSV パラメーターをもつコマンドレット	その CSV をインポートできるコマンドレット
dir -ExportCsv <file>	Import-Csv <file> New-Item
Get-OrchMachine -ExportCsv <file>	Import-Csv <file> New-OrchMachine
Get-OrchProcess -ExportCsv <file>	Import-Csv <file> New-OrchProcess Import-Csv <file> Update-OrchProcess
Get-OrchAsset -ExportCsv <file>	Import-Csv <file> Set-OrchAsset
Get-OrchAsset -ExportCredentialCsv <file>	Import-Csv <file> Set-OrchCredentialAsset
Get-OrchQueue -ExportCsv <file>	Import-Csv <file> New-OrchQueue Import-Csv <file> Update-OrchQueue
Get-PmGroup -ExportCsv <file>	Import-Csv <file> New-PmGroup
Get-PmRobotAccount -ExportCsv <file>	Import-Csv <file> Set-PmRobotAccount

※ テナントのすべてのエンティティを出力するには、上記の各 Get コマンドレットをルートフォルダーで、-Recurse パラメーターをつけて実行してください。

※ dir と New-Item は、カレントフォルダーが UiPathOrch ドライブでないと動作しないことにご注意ください。

CSV ファイルのインポート

UiPathOrch のほとんどのコマンドレットが、各パラメーターを CSV のインポートで指定できるようになっています。

一般に PowerShell のコマンドレットは、Import-Csv コマンドレットの出力をパイプでリダイレクトすることにより、各パラメーターの値を指定できます。たとえば、次のようにします。

```
PS> Import-Csv <file name> | Copy-OrchAsset
```

このとき、CSV ファイルの列値は、その列と同じ名前のパラメーターに渡されます。たとえば、次の.csv ファイルを Copy-OrchAsset コマンドレットにインポートすると

Path	Name	Destination
Orch1:¥Shared	アセット#1	Orch2:¥Shared
Orch1:¥Folder	アセット#2	Orch2:¥AnotherFolder

次のコマンドを順に実行すると同じ結果が得られます。

```
PS> Copy-OrchAsset -Path Orch1:¥Shared -Name アセット#1 -Destination Orch2:¥Shared  
PS> Copy-OrchAsset -Path Orch1:¥Folder -Name アセット#1 -Destination Orch2:¥Shared
```

なお、CSV ファイルの中に指定したパラメーター値は、コマンドレットのパラメーターで上書きできます。たとえば、次のコマンドは、CSV に記載されたアセットをすべてカレントフォルダーにインポートします。

```
PS Orch2:¥Shared> Import-Csv <file> | Set-OrchAsset -Path .
```

エンティティをコピーする

UiPathOrch には、エンティティをコピーするコマンドレットが多く含まれます。テナントエンティティをコピーするコマンドレットは、各エンティティを別のテナントにコピーします。フォルダーエンティティをコピーするコマンドレットは、各エンティティを同じテナントの別のフォルダーに、あるいは別のテナントのフォルダーにコピーします。

ここでは、Copy-OrchCalendar を例にして、使い方を説明します。ほかのコマンドレットも同じように使えます。

次のコマンドは、指定の名前のカレンダーを Orch1:ドライブから Orch2:ドライブにコピーします。

```
PS Orch1:¥> Copy-OrchCalendar <カレンダー名> Orch2:
```

名前には、ワイルドカードを含むテキストをカンマ区切りで複数指定できます。次のコマンドは、Orch1:ドライブのカレンダーをすべて Orch2:ドライブにコピーします。

```
PS Orch1:¥> Copy-OrchCalendar * Orch2:
```

カレントドライブが別のドライブのとき、コピー元を指定するには -Path パラメーターを使います。次のコマンドは、Orch1:ドライブのカレンダーをすべて Orch2:ドライブにコピーします。

```
PS C:¥> Copy-OrchCalendar -Path Orch1: * Orch2:
```

コピー元フォルダーの指定

コピー元フォルダーは、-Path パラメーターで指定します。-Path を指定しない場合、現在のフォルダーがコピー元となります。フォルダーエンティティをコピーするコマンドレットでは、-Path のほか -Recurse や -Depth パラメーターでコピー元を指定できます。これ

らのパラメーターを指定するときは、コマンドレット名の直後に指定すると便利です。ほかのパラメーターの前に指定することで、ほかのパラメーターの自動補完が適切に動作するようになります。

-Recurse パラメーターは、コピー元フォルダーとコピー先フォルダーが同じ構造のサブフォルダーをもつとき、コピー元のサブフォルダーにあるエンティティも、コピー先の対応するサブフォルダーにコピーします。

たとえば次のコマンドは、同じフォルダー構造をもつ Orch1:テナントと Orch2:テナントにおいて、Orch1:テナントにあるアセットをすべて Orch2:テナントにコピーします。

```
PS Orch1:¥> Copy-OrchAsset -Recurse * Orch2:¥
```

-Recurse パラメーターが実装されていないコマンドレットもあるのでご注意ください。なお、フォルダーエンティティをコピーすると、各エンティティに含まれるタグやリンクの情報もコピーされます。

フォルダーをコピーする

copy コマンド (Copy-Item コマンドレット) は、指定のフォルダーをコピーします。このとき、各フォルダーに含まれるフォルダーエンティティ (フォルダーパッケージやプロセス、アセット、キューなど) も自動でコピーします。

次のコマンドは、Orch1:テナントのフォルダーをすべて Orch2:テナントにコピーします。

```
PS Orch1:¥> copy -Recurse * Orch2:
```

なお、クラシックフォルダーをコピーすると、自動でモダンフォルダーに変換されます。

個人用ワークスペースをコピーする

自分以外の個人用ワークスペースも、探索を開始しておけば UiPathOrch で操作できます。探索の開始は Orchestrator API でサポートされていないため、Orchestrator Web で手動で行う必要があります。探索を開始したら、Clear-OrchCache コマンドレットを実行し、探索の開始を PowerShell コンソールに反映してください。

テナントを移行する

上記の説明したコピーのコマンドレットを使うと、簡単にテナントを移行できます。また、複数のテナントを同一のテナントに集約することも簡単です。CSV ファイルを作成する必

要はありません。ワイルドカードを使えば、すべてのエンティティを簡単に一括してコピーできます。

また、あるフォルダーのエンティティを別のフォルダーにコピーすることもできます。さらに、コピーするコマンドレットのそれぞれは CSV ファイルからのインポートをサポートしているので、どのエンティティをどのフォルダーにコピーするか、予め CSV ファイルで計画してから、それをインポートすることでコピーを完了することもできます。多くのコピーコマンドレットのパラメーターは Path,Name,Destination の 3 つなので、これらの列を CSV ファイルに記載してください。

テナント移行の手順

テナントを移行するには、次の 2 つのステップが必要となります。

1. 組織のエンティティを移行する
2. テナントエンティティとフォルダーエンティティを移行する

- なお、組織のエンティティには次が含まれます:
 - ローカルユーザー、ローカルグループ、ロボットアカウント
- テナントエンティティには次が含まれます:
 - ライブラリ、パッケージ、資格情報ストア、ロール、テナントに割り当てられたユーザーとロボット、マシン、カレンダー、Webhook
- フォルダーエンティティには次が含まれます:
 - フォルダー、フォルダーユーザー、フォルダーマシン、ストレージバケット、フォルダーフィールドのパッケージ、プロセス、アセット、キュー、トリガー、API トリガー、テストセット、テストスケジュール、テストデータキューとアイテム、アクションカタログ

以降では、Orch1: のエンティティを Orch2: に移行（コピー）するコマンド例を示します。

組織のエンティティを移行する手順

下記のふたつのコマンドレットを実行することでコピーできます。

```
PS Orch1:¥> Copy-PmUser * Orch2: -WhatIf
PS Orch1:¥> Copy-PmRobotAccount * Orch2: -WhatIf
```

- -WhatIf スイッチパラメーターは、実際にはコピーをせずにログだけを出力します。問題ないことを確認したら、-WhatIf を除去して再実行してください。
- Copy-PmUser は、指定のユーザーをコピーするとき、このユーザーを自動で同じ組

組織グループに追加します。同名の組織グループがコピー先の組織に存在しない場合には、自動で同名のグループを作成します。このため、組織のローカルグループを明示的にコピーする必要はありません。

- 組織が Active Directory に統合されている場合には、組織のローカルユーザーをコピーする必要はありません。
- ただし、組織のローカルグループにディレクトリユーザーが追加されている場合には、組織のローカルグループの移行が必要となります。次のようにして移行できます。

```
PS Orch1:¥> Get-PmGroupMember -ExportCsv c:groups.csv
PS Orch1:¥> Import-Csv c:groups.csv | Add-PmGroupMember -Path Orch2:
```

テナントエンティティとフォルダーエンティティを移行する手順

次のコマンドレットを実行することで、テナントエンティティとフォルダーエンティティの両方をコピーできます。

```
PS Orch1:¥> copy . Orch2:¥ -Recurse -WhatIf
```

- ルートフォルダーを別のテナントのルートフォルダーにコピーする操作をすると、すべてのテナントエンティティがコピーされます。この機能は、UiPathOrch version 0.9.11.0 で追加されました。
- フォルダーをコピーすると、そのフォルダーに含まれるすべてのフォルダーエンティティがコピーされます。
- -WhatIf スイッチパラメーターは、実際にはコピーをせずにログだけを出力します。問題ないことを確認したら、-WhatIf を除去して再実行してください。
- -WhatIf の代わりに-Confirm を指定すると、コピーするエンティティとコピーしないエンティティを簡単に指定できます。
- 個人用ワークスペースを移行する必要がある場合には、移行元と移行先のテナントの両方で、対象となる個人用ワークスペースに UiPathOrch でアクセスできるようになっている必要があります。Orchestrator web で個人用ワークスペースの探索を開始し、UiPathOrch の PowerShell ドライブで Clear-OrchCache コマンドレットを実行してください。dir コマンドを実行し、個人用ワークスペースフォルダーが表示されることを確認してください。個人用ワークスペースの探索開始は、Orchestrator Web API で行えないため、手動で開始する必要があります。ユーザー数が多いと大変ですが、がんばってください。
- なお、テナントにユーザーを作成した後に次を実行すると、まとめて個人用ワークスペースフォルダーを有効にできます。

```
PS> Enable-OrchPersonalWorkspace -Path Orch2: *
```

Copy-Item (別名: copy) の実行方法には、次のようなバリエーションがあります。

- テナントエンティティとフォルダーエンティティをすべてコピーする:

```
PS> copy Orch1:¥ Orch2:¥ -Recurse
```

- ある種類のテナントエンティティと、特定のフォルダーだけをコピーする:

```
PS> copy Orch1:¥ Orch2:¥ -Recurse -Confirm
```

- すべてのテナントエンティティをコピーする（フォルダーは除外する）:

```
PS> copy Orch1:¥ Orch2:¥
```

- すべてのフォルダーをコピーする（テナントエンティティは除外する）:

```
PS> copy Orch1:¥* Orch2:¥ -Recurse
```

Orch1:¥* には、Orch1:¥ は含まれないことに留意してください。

- すべてのフォルダーのみをコピーし、テナントエンティティとフォルダーに含まれるエンティティはコピーしない:

```
PS> copy Orch1:¥* Orch2:¥ -Recurse -ExcludeEntities
```

もし、特定のエンティティのみをコピーしたい場合には、エンティティの種別ごとに用意されているコピーコマンドレットをご利用ください。テナントを移行するときには、各エンティティを適切な順でコピーすることが重要です。たとえば、プロセスをコピーする前にパッケージをコピーしてください。パッケージが存在しないと、プロセスのコピーに失敗してしまうからです。テナントエンティティをすべてコピーしてからフォルダーをコピーすべきです。次の順で各コマンドレットを実行すると良いでしょう。

```
PS Orch1:¥> Copy-OrchLibrary * * Orch2:¥
```

```
PS Orch1:¥> Copy-OrchPackage * * Orch2:¥
```

```
PS Orch1:¥> Copy-OrchCredentialStore * Orch2:¥
```

```
PS Orch1:¥> Copy-OrchRole * Orch2:¥
```

```
PS Orch1:¥> Copy-OrchUser * Orch2:¥
```

```
PS Orch1:¥> Copy-OrchMachine * Orch2:¥
```

```
PS Orch1:¥> Copy-OrchCalendar * Orch2:¥
```

```
PS Orch1:¥> Copy-OrchWebhook * Orch2:¥
```

（コピーが必要な個人用ワークスペースについて、Orchestrator web で手動で探索を開始）

```
PS Orch1:¥> Clear-OrchCache
```

```
PS Orch1:¥> copy -Recurse * Orch2:¥
```

テナント移行時の注意事項

テナントを移行するときは、コピー元とコピー先のドライブ名を間違えないように特に注意してください。ドライブ名を Src と Dst のように、間違えようのない名前にしてから作業すると良いでしょう。また、Src ドライブのスコップはすべて Read のみとしておき、誤って Src ドライブのエントリを削除することがないようにしてください。

コマンドレットリファレンス

本稿では、コマンドレットの名前とパラメーターを次のように示します。

<i>Get-ChildItem</i> <i>[-Path]</i> <i><string[]></i> <i>[-Filter]</i> <i><string></i> <i>[-Include <string[]>]</i> <i>[-Exclude <string[]>]</i> <i>[-Recurse]</i> <i>[-Depth <uint>]</i> <i>[-Force]</i> <i>[-Reload]</i> 必要な権限: OR.Folders.Read
--

- 大かっこ [] は省略可能な部分です。パラメーター名が大かっこ [] で括弧されている場合、それは位置指定パラメーターなので、パラメーター名を省略できることを意味します。
- 中かっこ <> は、具体的な値で置換すべき部分です。たとえば <string> の部分には、具体的な string 値（テキスト）を指定する必要があります。
- <string[]> は、string 値（テキスト）の配列です。ここにはカンマ区切りで複数の値を指定できます。<uint> は符号なし整数（unsigned integer）です。
- パラメーター名のみを指定したもの（パラメーター値の指定がないもの）は、スイッチパラメーターです。上記の例では、-Recurse、-Force がスイッチパラメーターです。
- 必要な権限は、当該のコマンドレットの実行に際して設定ファイルで指定が必要となる権限です。たとえば OR.Folders.Read は Orchestrator のフォルダーを読み取る権限、OR.Folders.Write はフォルダーを書き込む権限です。OR.Folders は、読み取りと書き込みの両方の権限を内包する上位の権限です。なお、すべてのコマンドレットの実行に際して、少なくとも OR.Folders.Read が必要です。

フォルダーの操作

Orchestrator のフォルダーは、標準コマンドレットで操作します。使用頻度が低いパラメーターや共通パラメーターの説明は省略します。正確なパラメーターの一覧は、Microsoft のドキュメントを参照してください。

- ***Set-Location* ***[-Path]*** ***<string>*****
必要な権限: OR.Folders.Read

現在のロケーションを変更します。別名は cd です。

- ***Get-Location***

必要な権限: OR.Folders.Read

現在のロケーションを取得します。別名は pwd です。

- ***Get-ChildItem* *[-Path]* *<string[]>* *[-Filter]* *<string>* *[-Include]* *<string[]>* *[-Exclude]* *<string[]>* *[-Recurse]* *[-Depth]* *<uint>* *[-Force]* *[-Name]* *[-Reload]***

必要な権限: OR.Folders.Read

-Path で指定したフォルダーのサブフォルダーを表示します。-Path を指定しない場合、現在のフォルダーのサブフォルダーを表示します。

- ***Invoke-Item* *[-Path]* *<string[]>***

必要な権限: OR.Folders.Read

指定のロケーションをブラウザで開きます。別名は ii です。

- ***New-Item* *[-Path]* *<string[]>* *[-WhatIf]* *[-Confirm]* *[-Description]* *<string>* *[-FeedType]* *<OrchAPISession+FolderFeedType>***

必要な権限: OR.Folders.Write

フォルダーを作成します。別名は ni です。New-Item を呼び出す関数 mkdir もありますが、mkdir は別名ではないため、-Description パラメーターは利用できないことに注意してください。

- ***Remove-Item* *[-Path]* *<string[]>* *[-Filter]* *<string>* *[-Include]* *<string[]>* *[-Exclude]* *<string[]>* *[-Recurse]* *[-Force]* *[-WhatIf]* *[-Confirm]***

必要な権限: OR.Folders.Write

フォルダーを削除します。別名は rmdir です。

- ***Move-Item* *[-Path]* *<string[]>* *[-Destination]* *<string>* *[-Filter]* *<string>* *[-Include]* *<string[]>* *[-Exclude]* *<string[]>* *[-PassThru]* *[-WhatIf]* *[-Confirm]***

必要な権限: OR.Folders.Write

フォルダーを移動します。別名は mv です。

現在のロケーションを変更する

- ***cd Orch2:***

ドライブ名のみを指定した場合は、そのドライブのカレントロケーションに移動します。

- ***cd ¥***
カレントドライブのルートフォルダーに移動します。¥ はルートフォルダーです。cd / とすることもできます。
- ***cd Orch2:¥***
Orch2:のルートフォルダーに移動します。
- ***cd ..***
現在のフォルダーの親フォルダーに移動します。.. は親フォルダーです。
- ***cd ../..***
現在のフォルダーの親の親フォルダーに移動します。
- ***cd [Ctrl+Space]***
補完で現在のフォルダーのサブフォルダーを一覧表示し、選択して移動します。
- ***cd /[Ctrl+Space]***
/ を入力してから補完すると、(現在のフォルダーに関わらず) ルートフォルダーの直下にあるフォルダーが移動先の候補として表示されます。
- ***cd h****
名前が h で始まるサブフォルダーに移動します。* は、任意のテキストに合致するワイルドカードです。合致するフォルダーが複数ある場合にはエラーとなります。

フォルダーを表示する

- ***dir***
現在のフォルダーの直接のサブフォルダーを表示します。dir は Get-ChildItem の別名です。
- ***dir -Recurse***
現在のフォルダーの子孫のサブフォルダーをすべて表示します。ルートフォルダーで実行すると、テナントにあるすべてのフォルダーを表示します。
- ***dir -Depth 2***
現在のフォルダーの子フォルダーと孫フォルダーまでをすべて表示します。現在のフォルダーの Depth はゼロです。Orchestrator のモダンフォルダーの階層は最大で 7 なので、-Depth 6 は -Recurse と同じ動作になります。
- ***dir a****
a ではじまる名前のサブフォルダーを表示します。
- ***dir -Path Orch1:***
Orch1:のカレントロケーションのサブフォルダーを表示します。

- ***dir -Path Orch1:¥***
Orch1: のルートフォルダーのサブフォルダーを表示します。
- ***dir / ? { \$_.FeedType -eq 'FolderHierarchy' }***
フィード付きのフォルダーのみを表示します。
- ***dir / ? { \$_.ProvisionType -eq 'Manual' }***
クラシックフォルダーのみを表示します。
- ***gi . / Format-List***
カレントフォルダーの情報を表示します。gi は *Get-Item* の別名です。

フォルダーをブラウザーで開く

- ***ii .***
現在のフォルダーをブラウザーで開きます。.*は現在のフォルダー（カレントロケーション）です。*
- ***ii ...***
カンマで区切って、複数のフォルダーを同時に指定できます。上記の例では、現在のフォルダーとその親フォルダーを指定しています。

フォルダーを作成する

フォルダーの作成は *mkdir* で行えます。ただし、*mkdir* コマンドに *-Description* や *-FeedType* パラメーターは指定できません。これらを指定するには、*New-Item* を使います。

- ***mkdir hoge***
現在のフォルダーの下に、hoge という名前のフォルダーを作成します。
- ***mkdir f1,f2,f3***
現在のフォルダーの下に、f1、f2、f3 という名前でフォルダーを3つ作成します。
- ***New-Item fuga -Description 'This is a new folder.'***
現在のフォルダーの下に fuga という名前のフォルダーを作成し、その説明に This is a new folder. というテキストを設定します。
- ***New-Item ¥yyy -FeedType FolderHierarchy***
ルートフォルダーの直下に、yyy という名前でフィード付きフォルダーを作成します。*-FeedType* に指定できる値は補完で入力できます。フィード付きフォルダーは、ルートフォルダーの直下のみ作成できることに注意してください。

フォルダーをコピーする

フォルダーは、*copy* コマンドでコピーできます。*copy* は *Copy-Item* の別名です。同じテ

ナント内で、もしくは異なるテナント間でコピーできます。*copy* は、フォルダーに割り当てられたユーザー、ロール、マシンも合わせてコピーします。同じテナント内のフォルダーをコピーする場合、ユーザー、ロール、マシンの内部 ID はそのまま複製されます。しかし、異なるテナント間でコピーする場合、コマンドは対象テナント内でユーザー、ロール、マシンを名前を検索し、その ID をコピーしたフォルダーに割り当てます。対象テナントに同じ名前のエンティティが存在しない場合、コマンドは警告を表示しますが、コピー処理は継続します。

個人用ワークスペースフォルダーはコピーされません。クラシックフォルダーをコピーすると、コピー先のフォルダーはモダンフォルダーになります。ルートフォルダーの直下にあるフィードつきフォルダーは、同じドライブもしくは別のドライブのルートフォルダーの直下のみコピーできます。最上位でないフィードつきフォルダーは、任意のフォルダーの下にコピーできます。

- ***copy Orch1:¥* Orch2:¥ -Recurse***

Orch1:¥ のすべてのフォルダーとそのサブフォルダーを、Orch2:¥ にコピーします。

- ***copy srcFolder dstFolder***

現在のフォルダー直下にある srcFolder を、現在のフォルダー直下にある dstFolder にコピーして、dstFolder¥srcFolder フォルダーを作成します。

フォルダーを移動する

フォルダーの移動は、同じテナント間でのみ行えます。(異なるテナント間でのフォルダーの移動はサポートされません)

- ***mv hoge fuga***

現在のフォルダーの下にある hoge フォルダーを、fuga フォルダーの下に移動します。

- ***mv f1,f2 fuga***

現在のフォルダーの下にある f1 フォルダーと f2 フォルダーを、fuga フォルダーの下に移動します。

- ***mv * fuga***

現在のフォルダーの下にあるすべてのフォルダーを、fuga フォルダーの下に移動します。

- ***mv h* fuga***

現在のフォルダーの下にある h から始まるフォルダーをすべて、fuga フォルダーの下に移動します。

- ***mv h*..***

現在のフォルダーの下にある h から始まるフォルダーをすべて、親フォルダーの下に

移動します。

- ***mv h* fuga -WhatIf***

mv h* fuga を実行すると何が起こるかを確認します。実際にはフォルダーは移動されません。

フォルダーをリネームする

- ***ren hoge hoehoe***

現在のフォルダーの下にある hoge フォルダーの名前を hoehoe に変更します。

フォルダーを削除する

- ***rmdir hoge***

現在のフォルダーの下にある hoge フォルダーを削除します。

- ***rmdir ****

現在のフォルダーの下にあるフォルダーをすべて削除します。確認メッセージは表示せず、即時に削除します。この操作は元に戻せないので、注意して操作してください。

- ***rmdir h* -WhatIf***

現在のフォルダーの下にある h で始まるフォルダーをすべて削除します。-WhatIf パラメーターは、実際には削除せず、削除対象とするフォルダー名を確認することを指示します。

- ***rmdir h* -Confirm***

現在のフォルダーの下にある h で始まるフォルダーをすべて削除します。フォルダーをひとつ削除するたび、確認メッセージを表示します。

フォルダーの使用状況を表示する

Get-OrchFolderUsage コマンドレットは、非公開の (swagger ドキュメントに記載されていない) OC Web API を呼び出すことにより実装されているため、Orchestrator のバージョンによっては動作しない場合があります。

- ***Get-OrchFolderUsage***

現在のフォルダーのサマリ情報を表示します。

- ***Get-OrchFolderUsage -Recurse / Export-Csv c:¥temp¥usage.csv -Encoding sjis***

現在のフォルダーと、そのサブフォルダーのサマリ情報を、c:¥temp¥usage.csv ファイルに保存します。

- ***Get-OrchPersonalWorkspace / Get-OrchFolderUsage***

すべての個人用ワークスペースフォルダーのサマリ情報を表示します。自分以外の個

人用ワークスペースフォルダーは *dir* コマンドで表示されませんが、*Get-OrchPersonalWorkspace* の出力をパイプで *Get-OrchFolderUsage* に入力することで、そのサマリ情報を取得できます。

ライブラリパッケージの操作

- ***Get-OrchLibrary* *[-Id]* *<string[]>* *[-Path <string[]>]***

必要な権限: OR.Execution.Read

現在のドライブ（テナント）のテナントフィールドにあるライブラリの最新バージョンを取得します。最初のパラメーターで、ライブラリの名前をカンマ区切りとワイルドカードで複数指定できます。ライブラリの名前を省略すると、すべてのライブラリを列挙します。*-Path* パラメーターには、ドライブ名を指定します。

- ***Get-OrchLibraryVersion* *[-Id]* *<string[]>* *[-Path <string[]>]***

必要な権限: OR.Execution.Read

現在のドライブ（テナント）のテナントフィールドにあるライブラリのすべてのバージョンを取得します。最初のパラメーターで、ライブラリの名前をカンマ区切りとワイルドカードで複数指定できます。ライブラリの名前を省略すると、すべてのライブラリを列挙します。*-Path* パラメーターには、ドライブ名を指定します。

- ***Remove-OrchLibrary* *[-Id]* *<string[]>* *[-Version <string[]>]* *[-Path <string[]>]* *[-WhatIf]* *[-Confirm]***

必要な権限: OR.Execution.Write

現在のドライブ（テナント）のテナントフィールドにあるライブラリを削除します。最初のパラメーターで、ライブラリの名前を指定します。次のパラメーターで、削除するバージョンを指定します。これらのパラメーターには、ワイルドカードを含むテキストをカンマ区切りで複数指定できます。ライブラリの名前とバージョンは省略できません。バージョンに*を指定すると、すべてのバージョンを削除します。補完により、存在するライブラリとバージョンを表示できます。

- ***Import-OrchLibrary* *[-Source]* *<string[]>* *[-Path <string>]* *[-WhatIf]* *[-Confirm]***

必要な権限: OR.Execution.Write

指定のパッケージファイルを、テナントのライブラリフィールドにアップロードします。*-Source* パラメーターには、ローカルドライブの *nupkg* ファイル名もしくは *nupkg* ファイルを含むディレクトリ名をカンマ区切りで指定します。ワイルドカードを利用できます。

- ***Export-OrchLibrary* *[[-Id <string[]>] [-Version <string[]>] [-Destination <string>] [-Path <string[]>] [-WhatIf] [-Confirm]]***

必要な権限: OR.Execution.Read

指定のライブラリをダウンロードします。最初のパラメーターで、ライブラリの名前を指定します。次のパラメーターで、削除するバージョンを指定します。これらのパラメーターには、ワイルドカードを含むテキストをカンマ区切りで複数指定できます。補完により、存在するライブラリとバージョンを表示できます。これらのパラメーターを省略すると、すべてのライブラリをダウンロードします。

ライブラリの最新バージョンを表示する

- ***Get-OrchLibrary***

現在のドライブ（テナント）のライブラリフィードにあるパッケージをすべて表示します。

- ***Get-OrchLibrary -Path Orch1;Orch2:***

Orch1:ドライブと Orch2:ドライブのライブラリフィードにあるパッケージをすべて表示します。ドライブ名は補完入力できます。

- ***Get-OrchLibrary e****

現在のドライブ（テナント）のライブラリフィードにある、e で始まる名前のパッケージをすべて表示します。パッケージ名は補完入力できます。また、カンマで区切って複数のパッケージ名を指定できます。

ライブラリのすべてのバージョンを表示する

- ***Get-OrchLibraryVersion***

現在のドライブ（テナント）のライブラリフィードにあるパッケージのすべてのバージョンを表示します。

- ***Get-OrchLibraryVersion -Path Orch1;Orch2:***

Orch1:ドライブと Orch2:ドライブのライブラリフィードにあるパッケージのすべてのバージョンを表示します。ドライブ名は補完入力できます。

- ***Get-OrchLibraryVersion e****

現在のドライブ（テナント）のライブラリフィードにある、e で始まる名前のパッケージのすべてのバージョンを表示します。パッケージ名は補完入力できます。また、カンマで区切って複数のパッケージ名を指定できます。

ライブラリを削除する

使用されているライブラリパッケージであっても、下記の実行により削除できることに注意してください。

- ***Remove-OrchLibrary <パッケージ名> <バージョン番号>***

現在のドライブ（テナント）のライブラリフィールドにある、指定のライブラリ名の指定のバージョンのパッケージを削除します。パッケージ名とバージョン番号は省略できません。補完機能で入力してください。パッケージ名とバージョン番号にはワイルドカードが使えます。

- ***Remove-OrchLibrary *mylib** -WhatIf***

現在のドライブ（テナント）のライブラリフィールドにある、名前に mylib を含むライブラリのすべてのバージョンを削除します。実際には削除せず、削除対象となるライブラリとバージョンを表示します。

- ***Remove-OrchLibrary *mylib** -Confirm***

現在のドライブ（テナント）のライブラリフィールドにある、名前に mylib を含むライブラリのすべてのバージョンを削除します。ひとつ削除するたび、確認メッセージを表示します。

ライブラリをアップロードする

- ***Import-OrchLibrary c:%pkg%*.nupkg***

指定のパッケージファイルを、現在のドライブのライブラリのテナントフィールドにアップロードします。

- ***Import-OrchLibrary c:%pkg***

-Source パラメーターにフォルダーを指定した場合には、このフォルダーに含まれる *.nupkg ファイルをすべてアップロードします。なお、-Source パラメーターには複数のフォルダー名とファイル名をカンマ区切りで同時に指定できます。

- ***Import-OrchLibrary c:%pkg%*.nupkg -Path Orch1;,Orch2:***

指定のパッケージファイルを、Orch1:と Orch2:のライブラリのテナントフィールドの両方にアップロードします。

ライブラリをダウンロードする

- ***Export-OrchLibrary***

現在のドライブ（テナント）のライブラリをすべてダウンロードし、C:ドライブのカレントロケーションに保存します。なお、C:ドライブのカレントロケーションをファ

イルエクスプローラーで開くには、*ii c:* とします。

- ***Export-OrchLibrary* <ライブラリ名>**
指定のライブラリのすべてのバージョンをダウンロードし、C:ドライブのカレントロケーションに保存します。
- ***Export-OrchLibrary* <ライブラリ名> <バージョン番号>**
指定のライブラリの指定のバージョンをダウンロードし、C:ドライブのカレントロケーションに保存します。
- ***Export-OrchLibrary* <ライブラリ名> -Destination c:¥lib**
指定のライブラリのすべてのバージョンをダウンロードし、C:¥lib に保存します。

パッケージの操作

パッケージは、テナントのプロセスフィールドのほか、フィールド付きのフォルダーに配置されています。フィールド付きのフォルダー（とそのサブフォルダー）で *Get-OrchPackage* を実行すると、そのフォルダーフィールドのパッケージを表示します。フィールドがないフォルダー、もしくはルートフォルダーで *Get-OrchPackage* を実行した場合には、テナントのプロセスフィールドにあるパッケージを表示します。

- ***Get-OrchPackage* [[-Id] <string[]>] [-Path <string[]>] [-Recurse]**
必要な権限: OR.Execution.Read
パッケージの最新バージョンを取得します。
- ***Get-OrchPackageVersion* [[-Id] <string[]>] [-Path <string[]>] [-Recurse]**
必要な権限: OR.Execution.Read
パッケージのすべてのバージョンを取得します。
- ***Remove-OrchPackage* [-Id] <string[]> [-Version] <string[]> [-Path <string[]>] [-Recurse] [-WhatIf] [-Confirm]**
必要な権限: OR.Execution.Write
パッケージの指定のバージョンを削除します。
- ***Export-OrchPackage* [[-Id] <string[]>] [[-Version] <string[]>] [[-Destination] <string>] [-Path <string[]>] [-Recurse] [-WhatIf] [-Confirm]**
必要な権限: OR.Execution.Read
パッケージの指定のバージョンをダウンロードします。

現在のフォルダーで利用可能なパッケージを表示する

- ***Get-OrchPackage***

ルートフォルダー、もしくはフォルダーフィールドがないフォルダーで ***Get-OrchPackage*** を実行すると、そのテナントフィールドのパッケージを表示します。

フォルダーフィールド付きのフォルダーで ***Get-OrchPackage*** を実行すると、そのフォルダーフィールドのパッケージを表示します。

- ***Get-OrchPackage <パッケージ名>***

パッケージ名を、ワイルドカードとカンマ区切りで複数指定できます。補完入力でも指定できます。指定したパッケージのみを表示します。

現在のテナントのテナントフィールドにあるパッケージを表示する

- ***Get-OrchPackage -Path ¥***

現在のフォルダーにフィールドがあるかどうかに関係なく、テナントフィールドのパッケージを表示するには、-Path パラメーターにルートフォルダーを指定します。

現在のドライブのすべてのフィールドのパッケージを表示する

- ***Get-OrchPackage -Recurse***

ルートフォルダーで、-Recurse パラメーターを指定して Get-OrchPackage を実行すると、このテナントフィールドと、フィールド付きフォルダーにあるパッケージをすべて表示します。

- ***Get-OrchPackage -Path ****

ルートフォルダーで -Path * を指定して Get-OrchPackage を実行しても、上記と同じ結果が得られます。これは、フィールド付きのフォルダーを作成できるのはルートフォルダーの直下のみだからです。

- ***Get-OrchPackage -Recurse a****

-Recurse と合わせてパッケージ名を指定すると、この名前のパッケージを含むフォルダーフィールドを検索できます。-Recurse の後で補完入力すると、候補にはすべてのフィールドにあるパッケージ名が候補に表示されます。

パッケージのすべてのバージョンを表示する

- ***Get-OrchPackageVersion***

現在のフォルダーで利用可能なフィールドにあるパッケージのすべてのバージョンを表示します。

- ***Get-OrchPackageVersion -Recurse e****

現在のドライブ（テナント）のパッケージフィールドとフォルダーフィールドにある、eで始まる名前のパッケージのすべてのバージョンを表示します。パッケージ名は補完入力できます。また、カンマで区切って複数のパッケージ名を指定できます。

- ***Get-OrchPackageVersion -Path Orch1:¥,Orch2:¥ -Recurse***

Orch1:ドライブと Orch2:ドライブのすべてのフィールドにあるパッケージのすべてのバージョンを表示します。ドライブ名は補完入力できます。

パッケージを削除する

- ***Remove-OrchPackage <パッケージ名> <バージョン>***

現在のフォルダーで利用可能なフィールドから、指定のパッケージのバージョンを削除します。ワイルドカードを含むパッケージ名とバージョンをカンマ区切りで複数指定できます。補完もできます。なおアクティブなパッケージバージョンは、補完の候補リストに表示されません。

- ***Remove-OrchPackage -Recurse ** -WhatIf***

パッケージ名とバージョン番号の両方に*を指定すると、非アクティブのパッケージをすべてまとめて削除します。ルートフォルダーで-Recurse パラメーターを指定すると、テナントフィールドとすべてのフォルダーフィールドをまとめて処理します。-WhatIf で削除対象のパッケージを確認の上で、-WhatIf を指定せずに実行してください。

パッケージをアップロードする

- ***Import-OrchPackage c:¥pkg¥*.nupkg***

指定のパッケージファイルを、現在のフォルダーのプロセスフィールドにアップロードします。現在のフォルダーにプロセスフィールドがない場合は、テナントのパッケージフィールドにアップロードします。

- ***Import-OrchPackage c:¥pkg***

-Source パラメーターにフォルダーを指定した場合には、このフォルダーに含まれる*.nupkg ファイルをすべてアップロードします。なお、-Source パラメータには複数のフォルダー名とファイル名をカンマ区切りで同時に指定できます。

- ***Import-OrchPackage c:¥pkg¥*.nupkg -Path Orch1:¥,Orch2:¥myfolder***

指定のパッケージファイルを、Orch1:のテナントプロセスフィールドと Orch2:のmyfolder のプロセスフィールドの両方にアップロードします。Orch2:¥myfolder フォルダーにプロセスフィールドがない場合は、Orch2:のテナントプロセスフィールドにアップロードします。

- ***Import-OrchPackage c:¥pkg -Recurse***

ルートフォルダーで-Recurse パラメーターをつけて実行すると、テナントフィールドとすべてのフィールドつきフォルダーに一括してアップロードします。なお、フィールドを付与できるフォルダーはルートフォルダー直下のフォルダーだけなので、-Recurse オプションを指定するときには、ルートフォルダーで実行するときのみ意味をもちます。

- ***Import-OrchPackage c:¥pkg -SourceRecurse***

-SourceRecurse スイッチパラメーターを指定すると、-Source に指定したフォルダー（この例では c:¥pkg）のサブフォルダーにある*.nupkg ファイルを、同名のモダンフォルダーのフィールドにアップロードします。このパラメーターは、個人用ワークスペースフォルダーやフィールドつきのフォルダーと同名のフォルダーが、c:¥pkg の下にあることを前提として動作します。Export-OrchPackage -Recurse でダウンロードしたパッケージを、一括して別のテナントに移行するときに便利です。

パッケージをダウンロードする

- ***Export-OrchPackage***

現在のフォルダーのフィールドからすべてのパッケージをダウンロードして、C:ドライブのカレントロケーションに保存します。なお、C:ドライブのカレントロケーションをファイルエクスプローラーで開くには、*ii c:* とします。なお、フィールドつきフォルダーからパッケージをダウンロードするときは、同名のサブフォルダーがダウンロード先のフォルダーに自動で作成されます。

- ***Export-OrchPackage -Recurse***

ルートフォルダーで実行すると、テナントフィールドのすべてのパッケージと、すべてのフィールドつきフォルダーのすべてのパッケージをダウンロードします。

- ***Export-OrchPackage <プロセス名> <バージョン番号> -Destination c:¥pkg***

指定のプロセスの指定のバージョン番号を、c:¥pkg にダウンロードします。

- ***Export-OrchPackage * 1.0.1 -WhatIf***

すべてのパッケージのバージョン 1.0.1 を、C:のカレントロケーションにダウンロードします。バージョンに 1.0.[1-3]を指定すると、すべてのパッケージのバージョン 1.0.1,1.0.2,1.0.3 をダウンロードします。-WhatIf は、実際にはダウンロードせず、ダウンロード対象とするパッケージを確認することを指示します。

プロセスの操作

- ***Get-OrchProcess [[-Name] <string[]>] [-Path <string[]>] [-Recurse] [-Depth <uint>]***

必要な権限: OR.Execution.Read.

フォルダーに割り当てられて実行可能となっているプロセスを取得します。

- ***Remove-OrchProcess*** *[[**-Name**] <string[]>] [**-Path** <string[]>] [**-Recurse**] [**-Depth** <uint>] [**-WhatIf**] [**-Confirm**]*

必要な権限: OR.Execution.Write

フォルダーに割り当てられたプロセスを、フォルダーから削除します。

- ***Edit-OrchProcess*** *[[**-Name**] <string[]>] [**-Path** <string[]>]*

必要な権限: OR.Execution.Write

指定のプロセスを、ブラウザで編集します。

- ***Update-OrchProcessVersion*** *[[**-Name**] <string[]>] [**-Version** <string>] [**-Path** <string[]>] [**-Recurse**] [**-Depth** <uint>] [**-WhatIf**] [**-Confirm**]*

必要な権限: OR.Execution.Write

フォルダーに割り当てられたプロセスのバージョンを更新します。バージョンを指定しない場合は最新バージョンに更新します。

- ***Reset-OrchProcessVersion*** *[[**-Name**] <string[]>] [**-Path** <string[]>] [**-Recurse**] [**-Depth** <uint>] [**-WhatIf**] [**-Confirm**]*

必要な権限: OR.Execution.Write

フォルダーに割り当てられたプロセスのバージョンをロールバックします。

フォルダーに割り当てられたプロセスを表示する

- ***Get-OrchProcess***

現在のフォルダーに割り当てられたプロセスを表示します。

- ***Get-OrchProcess -Recurse***

現在のフォルダーと、そのすべてのサブフォルダーに割り当てられたプロセスを表示します。ルートフォルダーで実行すると、そのテナントに含まれるフォルダーすべてのプロセスを表示します。

- ***Get-OrchProcess -Recurse *proc****

プロセス名に proc を含むプロセスを、すべてのフォルダーから検索します。

プロセスをフォルダーから削除する

- ***Remove-OrchProcess*** <プロセス名>

指定のプロセスを、現在のフォルダーから削除します。プロセス名は補完入力できます。また、カンマで区切って複数を同時に指定できます。

- ***Remove-OrchProcess a****

現在のフォルダーから、名前が a で始まるプロセスをすべて削除します。

- ***Remove-OrchProcess a* -WhatIf***

実際に削除することなく、削除対象となるプロセスを表示します。

- ***Remove-OrchProcess -Recurse <プロセス名>***

指定のプロセスを、現在とそのすべてのサブフォルダーから削除します。プロセス名にはワイルドカードが使えます。

プロセスをブラウザーで編集する

- ***Edit-OrchProcess <プロセス名>***

指定のプロセスを、ブラウザーで開いて編集します。

フォルダーに割り当てられたプロセスのバージョンを更新する

- ***Update-OrchProcessVersion <プロセス名>***

現在のフォルダーで、指定のプロセスを最新バージョンに更新します。プロセス名には、ワイルドカードを含む名前をカンマ区切りで複数指定できます。補完入力もできます。補完で候補に表示されるのは、最新ではないプロセスのみです。

- ***Update-OrchProcessVersion <プロセス名> <バージョン>***

現在のフォルダーで、指定のプロセスを指定のバージョンに更新します。

- ***Update-OrchProcessVersion *-WhatIf***

現在のフォルダーで、すべてのプロセスを最新バージョンに更新します。-WhatIf パラメーターは、実際には更新せず、更新対象となるプロセスを表示することを指示します。

- ***Update-OrchProcessVersion -Recurse ****

現在のフォルダーとそのすべてのサブフォルダーで、すべてのプロセスを最新バージョンに更新します。ルートフォルダーで実行すると、すべてのフォルダーにおいて最新バージョンが利用可能となっているプロセスをすべて更新します。

- ***Update-OrchProcessVersion -Recurse <プロセス名> <バージョン>***

現在のフォルダーとそのすべてのサブフォルダーで、指定のプロセスを指定のバージョンに更新します。

- ***Reset-OrchProcessVersion <プロセス名>***

現在のフォルダーで、指定のプロセスのバージョンをロールバックします。プロセス名には、ワイルドカードを含む名前をカンマ区切りで複数指定できます。補完入力も

できます。

- ***Reset-OrchProcessVersion -Depth 1 <プロセス名>***

現在のフォルダーとその直接のサブフォルダーで、指定のプロセスのバージョンをロールバックします。

個人用ワークスペースの操作

非機密アプリの設定を使って接続したドライブにおいては、*dir* コマンド (*Get-ChildItem* コマンドレット) は接続ユーザーの個人用ワークスペースを自動で表示します。また、Orchestrator の web 画面で探索を有効にした (他人の) 個人用ワークスペースも、*dir* コマンドにより表示されます。*dir* コマンドが表示した個人用ワークスペースフォルダーは、ほかのフォルダーと同じように操作できます。たとえば、パッケージのアップロードとダウンロード、アセットの取得や更新などが行えます。なお、Orchestrator Web API は個人用ワークスペースの探索の開始/終了をサポートしません。他人の個人用ワークスペースを操作したいときは、まず Orchestrator の web 画面で、手動で探索を開始してください。その後、PowerShell コンソールで *Clear-OrchCache* を実行したあと、*dir* コマンドを実行すると、探索を開始した個人用ワークスペースが表示されます。

- ***Get-OrchPersonalWorkspace [-Name] <string[]> [-Path <string[]>]***

必要な権限: OR.Folders.Read.

個人用ワークスペースフォルダーを取得します。

個人用ワークスペースを表示する

- ***Get-OrchPersonalWorkspace***

このテナントの、すべての個人用ワークスペースを表示します。探索を開始していない個人用ワークスペースも表示します。しかし、探索を開始していない個人用ワークスペースの内容にアクセスすることはできません。

- ***Get-OrchPersonalWorkspace *name****

Name が *name* に合致する個人用ワークスペースを表示します。

ライセンスの操作

- ***Get-OrchLicenseNamedUser [-RobotType] <string[]> [-Path <string[]>]***

必要な権限: OR.License.Read

Named User License を取得します。-RobotType には、NonProduction、Attended、Unattended、Developmentなどを指定します。ワイルドカードを含むテキストをカン

マ区切りで複数指定できます。

- ***Get-OrchLicenseRuntime* *[-RobotType]* *<string[]>* *[-Path <string[]>]***

必要な権限: OR.License.Read

Runtime License を取得します。Named User License を取得します。*-RobotType* には、NonProduction、Attended、Unattended、Developmentなどを指定します。ワイルドカードを含むテキストをカンマ区切りで複数指定できます。

- ***Enable-OrchLicenseRuntime* *[-RobotType]* *<string[]>* *[-Key <string[]>]* *[-Path <string[]>]* *[-WhatIf]* *[-Confirm]***

必要な権限: OR.License.Write

Runtime License を有効化します。

- ***Disable-OrchLicenseRuntime* *[-RobotType]* *<string[]>* *[-Key <string[]>]* *[-Path <string[]>]* *[-WhatIf]* *[-Confirm]***

必要な権限: OR.License.Write

Runtime License を無効化します。

Named User License を表示する

- ***Get-OrchLicenseNamedUser***

このテナントの、すべての Named User License を表示します。

- ***Get-OrchLicenseNamedUser NonProduction***

NonProduction の Named User License を表示します。

- ***Get-OrchLicenseNamedUser NonProduction,Attended***

NonProduction と Attended の Named User License を表示します。

- ***Get-OrchLicenseNamedUser -Path Orch1: * / Export-Csv c:\nu.csv***

このテナントのすべての Named User License を、c:ドライブのカレントロケーションの nu.csv に出力します。このファイルを開くには、*c:\nu[Tab][Enter]* とします。あるいは、*ii c:[Enter]* として c:のカレントフォルダーをエクスプローラーで開きます。

- ***Get-OrchLicenseNamedUser -Path Orch1;,Orch2 ****

指定のドライブ（テナント）の、すべての Named User License を表示します。

Runtime License を表示する

- ***Get-OrchLicenseRuntime***

このテナントの、すべての Runtime License を表示します。

- ***Get-OrchLicenseRuntime NonProduction***

NonProduction の Runtime License を表示します。

- ***Get-OrchLicenseRuntime NonProduction,Attended***

NonProduction と Attended の Runtime License を表示します。

- ***Get-OrchLicenseRuntime -Path Orch1: * / Export-Csv c:lr.csv***

このテナントのすべての Runtime License を、c:ドライブのカレントロケーションの lr.csv に出力します。このファイルを開くには、*c:lr[Tab][Enter]* とします。あるいは、*ii c:[Enter]* として c:のカレントフォルダーをエクスプローラーで開きます。

- ***Get-OrchLicenseRuntime -Path Orch1;,Orch2 ****

指定のドライブ（テナント）の、すべての Runtime License を表示します。

Runtime Licensee を有効にする

- ***Enable-OrchLicenseRuntime Unattended <キー>***

RobotType が Unattended の<キー>のライセンスを有効にします。RobotType とキーは、ワイルドカードを含むテキストをカンマ区切りで複数指定できます。補完入力もできます。

- ***Enable-OrchLicenseRuntime **-WhatIf***

すべての RobotType で無効になっているライセンスをすべて有効化します。-WhatIf は、実際には有効化せず、有効化の対象とする RobotType とキーを確認することを指示します。

Runtime Licensee を無効にする

- ***Disable-OrchLicenseRuntime Unattended <キー>***

RobotType が Unattended の<キー>のライセンスを無効にします。RobotType とキーは、ワイルドカードを含むテキストをカンマ区切りで複数指定できます。補完入力もできます。

- ***Disable-OrchLicenseRuntime **-WhatIf***

すべての RobotType で有効となっているライセンスをすべて無効化します。-WhatIf は、実際には無効化せず、無効化の対象とする RobotType とキーを確認することを指示します。

ジョブの操作

UiPathOrch のインストールディレクトリ

%UserProfile%\Documents\PowerShell\Modules\UiPathOrch の下にある Examples ディレクトリには、15 分以上 Pending となっているジョブの存在を 15 分おきに検索し、あればメールで通知するサンプルスクリプトがあります。参考にしてください。

- ***Get-OrchJob [-Last <string>] [-CreationTimeAfter <datetime>] [-CreationTimeBefore <datetime>] [-Priority <string>] [-Process <string[]>] [-SourceType <string[]>] [-State <string[]>] [-Skip <ulong>] [-First <ulong>] [-Path <string[]>] [-Recurse] [-Depth <uint>]***
- ***Get-OrchJob [[-Id] <long[]>] [-Skip <ulong>] [-First <ulong>] [-Path <string[]>] [-Recurse] [-Depth <uint>]***

必要な権限: OR.Jobs.Read

ジョブを取得します。最初のパラメーターセットでは、さまざまなパラメーターで取得するジョブの条件を指定できます。ふたつめのパラメーターセットの -Id パラメーターには、取得するジョブの Id を直接指定できます。この -Id パラメーターの補完リストに表示されるのは、いちど取得して UiPathOrch のメモリ内にキャッシュされたジョブの Id のみです。そのため、最初のパラメーターセットで複数のジョブを取得した後、詳細に確認したいジョブがあれば、その Id を -Id パラメーターに指定してください。キャッシュ内に Id が存在する場合でも、Get-OrchJob はそのジョブの最新の状態を確認して表示します。

- ***Start-OrchJob [-Name] <string[]> [[-RuntimeType] <string>] [[-JobsCount] <int>] [-Path <string[]>] [-WhatIf] [-Confirm]***

必要な権限: OR.Jobs.Write

ジョブを開始します。-Name パラメーターの補完では、フォルダーに割り当てられているプロセスを指定できます。なお、複雑な条件（パラメーターやエントリポイントなど）は指定できません。

- ***Stop-OrchJob [-Id] <Object[]> [-Force] [-Path <string[]>]***

必要な権限: OR.Jobs.Write

ジョブを停止します。-Id パラメーターには、Get-OrchJob で確認したジョブの Id を指定してください。-Force スイッチパラメーターをつけると、指定のジョブを Kill します。

- ***Open-OrchJob [[-Id] <long[]>] [-Path <string[]>]***

必要な権限: OR.Job.Read

指定のジョブを、ブラウザで開きます。

ジョブを表示する

下記のパラメーターは、すべて同時に指定することもできます。

- ***Get-OrchJob***

現在のフォルダーのジョブを表示します。*Get-OrchJob* は、次回以降のコマンドレット実行時の補完入力のためにジョブ ID をキャッシュしますが、このキャッシュは *Get-OrchJob* の実行結果の出力には利用されません。最新の情報を表示するため、*Get-OrchJob* は必ず Orchestrator に問い合わせを実行します。

なお、*Get-OrchJob* が出力する日時データ (CreationTime、StartTime など) は UTC ですが、既定のテーブルビューがそれらをローカル時刻に変換して表示します。このビュー定義は、UiPathOrch モジュールのインストールディレクトリの OrchProvider.Format.ps1xml ファイルにあります。

- ***Get-OrchJob -Recurse***

現在のフォルダーとそのサブフォルダーのジョブをすべて表示します。

- ***Get-OrchJob -Path ..***

親フォルダーのジョブを表示します。

- ***Get-OrchJob -Recurse***

現在のフォルダーと、そのサブフォルダーのジョブをすべて表示します。

- ***Get-OrchJob -Last Day***

現在のフォルダーの過去 1 日間のジョブを表示します。-Last パラメーターの値には、Hour、Day、Week、Monthなどを補完で指定できます。このほか、

- ***Get-OrchJob -State Pending,Suspended***

ステートが保留中もしくは中断となっているジョブを表示します。失敗したジョブだけを表示するには、-State パラメーターに Faulted を指定します。このパラメーター値は補完入力できます。

- ***Get-OrchJob -SourceType QueueTrigger***

キュートリガーにより開始されたジョブを表示します。

- ***Get-OrchJob -Process <プロセス名>***

指定のプロセスのジョブを表示します。プロセス名にはワイルドカードが使えます。
*を指定すると、そのフォルダーに割り当てられているすべてのプロセスのログを表示します。プロセス名は、カンマ区切りで複数を同時に指定することもできます。-Process パラメーターの補完機能は、このフォルダーに割り当てられているプロセスの名前を一覧表示します。

このほか、-CreationTimeAfter、-CreationTimeBefore、-Priorityなどのパラメーターで表示するジョブをフィルターできます。

- ***Get-OrchJob | ? { \$_.CreationTime.ToLocalTime() -ge '2024/01/15 14:00:00' }***

-CreationTimeAfter と -CreationTimeBefore に指定する日時データは、ローカル時刻として解釈されます。しかし *Get-OrchJob* が出力する CreationTime は UTC なので、パイプで? (Where-Object の別名) にリダイレクトして CreationTime でフィルター

するときには、上記のように手動でローカル時刻に変換してから比較してください。
Get-OrchJob の出力結果を表示する既定のテーブルビューは、UTC をローカル時刻に変換してから表示することにも注意してください。

- ***Get-OrchJob / ? StopStrategy -eq Kill***
強制終了されたジョブのみを表示します。

ジョブをグループ化して数を数える

- ***Get-OrchJob / group ReleaseName -NoElement***
プロセス名でグループ化します。項目名は補完入力できます。Group は Group-Object の別名です。
- ***Get-OrchJob / group HostMachineName -NoElement***
マシン名でグループ化します。
- ***Get-OrchJob / group HostMachineName,State -NoElement***
マシン名とステータスでグループ化します。
- ***Get-OrchJob / group LocalSystemAccount,State -NoElement***
実行したアカウント名でグループ化します。
- ***Get-OrchJob / group { \$_.StartTime.ToLocalTime().Date } -NoElement***
開始日でグループ化します。
- ***Get-OrchJob / group { \$_.StartTime.ToLocalTime().ToString('yyyy/MM/dd') }***
開始月でグループ化します。
- ***Get-OrchJob / group { \$_.StartTime.ToLocalTime().DayOfWeek }***
開始曜日でグループ化します。
- ***Get-OrchJob / group { \$_.StartTime.ToLocalTime().DayOfWeek },State***
開始曜日とステータスでグループ化します。
- ***Get-OrchJob / ? { \$_.StartTime.ToLocalTime().DayOfWeek -eq 'Sunday' }***
日曜日に実行したジョブだけを出力します。
- ***Get-OrchJob / group { \$_.StartTime.ToLocalTime().Hour },State***
開始した時間帯とステータスでグループ化します。ジョブがどの時間帯で実行されていたか、ステートごとに傾向を簡単に分析できます。
- ***Get-OrchJob / group ReleaseName -NoElement / sort Count -Descending***
グループ化した内容を、数が大きい順に表示します。sort は Sort-Object の別名です。
- ***Get-OrchJob -State Faulted / group ReleaseName / sort Count -Descending***
失敗したジョブだけをプロセス名でグループ化し、数が大きい順に表示します。より頻繁に失敗しているプロセスはどれか、簡単に分析できます。

各ジョブの実行時間を求める

- ***Get-OrchJob / sort { (\$_.EndTime - \$_.StartTime).TotalSeconds } -Descending***
ジョブを、実行時間が長い順に並べます。
- ***Get-OrchJob / select Id, ReleaseName, @{Name='TotalSeconds'; Expression={\$_.EndTime - \$_.StartTime}.TotalSeconds } / sort TotalSeconds -Descending***
ジョブの詳細とあわせて、その実行時間も出力します。
- ***Get-OrchJob / select *, @{Name='TotalSeconds'; Expression={\$_.EndTime - \$_.StartTime}.TotalSeconds } / sort TotalSeconds***
計算した実行時間と合わせて、すべての列も出力します。列名の指定にはワイルドカードが使えるため、*を指定するとすべての列名が出力されます。
- ***Get-OrchJob / % { (\$_.EndTime - \$_.StartTime).TotalMinutes } / Measure-Object -Average -Sum -Maximum -Minimum***
ジョブの実行時間の平均、最長、最短を出力します。% は ForEach-Object の別名です。

ジョブを開始する

- ***Start-OrchJob <プロセス名>***
現在のフォルダーで、ジョブを開始します。プロセス名は補完入力できます。
- ***Start-OrchJob <プロセス名> Unattended***
プロセス名の直後に、ライセンスの種別 (-RuntimeType パラメーター) を指定できます。上記の例は、指定のプロセスを Unattended ライセンスで実行します。
- ***Start-OrchJob <プロセス名> Unattended 3***
実行を開始するジョブの数 (-JobsCount パラメーター) を指定できます。
- ***Start-OrchJob <プロセス名> -JobsCount 3***
ライセンスの種別を指定せず、ジョブの数だけを指定したい場合には、-JobsCount パラメーターの名前を指定してください。

ジョブを停止する

- ***Stop-OrchJob <ジョブID>***
指定のジョブ ID のジョブを停止します。ジョブ ID (-Id パラメーター) の補完入力により表示されるジョブ ID は、Get-OrchJob で取得済み（キャッシュ済み）で、かつ停止可能な（実行中の）ジョブの ID のみです。なお、補完入力を使わずにジョブ

ID を手入力した場合は、そのジョブ ID がキャッシュされていない状態でも問題なく動作します。

- ***Stop-OrchJob <ジョブ ID>, <ジョブ ID> -Force***

ジョブを強制終了するには、-Force スイッチパラメーターを指定します。ジョブ ID はカンマ区切りで複数を同時に指定できますが、ワイルドカードはサポートしないことに注意してください。

- ***Get-OrchJob / Stop-OrchJob***

Stop-OrchJob は、Get-OrchJob からのパイプ入力を受け入れます。これにより、現在のフォルダーで実行中のジョブをすべて停止できます。ジョブ ID を指定する必要がないので便利です。

- ***Get-OrchJob -Process <プロセス名> / Stop-OrchJob -WhatIf***

指定のプロセス名のジョブをすべて停止します。-WhatIf をつけると、実際に停止することなく、停止対象のジョブ ID を確認できます。

- ***Get-OrchJob -Recurse / Stop-OrchJob -Force***

現在のフォルダーと、そのサブフォルダーすべてにおいて実行中のジョブをすべて強制終了します。Stop-OrchJob には-Recurse オプションはありませんが、Get-OrchJob -Recurse の出力を Stop-OrchJob にパイプ入力することで、サブフォルダーすべてで実行中のジョブをまとめて停止できます。

ジョブをブラウザーで開く

- ***Open-OrchJob <ジョブ ID>***

指定のジョブを、ブラウザーで開きます。

ジョブ記録の操作

- ***Get-OrchJobVideo [-Skip <int>] [-First <int>] [-Path <string[]>] [-Recurse] [-Depth <uint>]***

必要な権限: OR.Job.Read

ビデオが添付されたジョブを取得します。

- ***Get-OrchJobMedia [-Skip <ulong>] [-First <ulong>] [-Path <string[]>] [-Recurse] [-Depth <uint>]***

必要な権限: OR.Monitoring.Read

スクリーンショットが添付されたジョブを取得します。

- ***Export-OrchJobMedia [[-JobId] <long[]>] [[-Destination] <string>] [-Path <string[]>] [-Recurse] [-Depth <uint>] [-WhatIf] [-Confirm]***

必要な権限: OR.Monitoring.Read

ジョブに添付されたスクリーンショットをダウンロードして、ファイルに保存します。
-Destination パラメーターには、保存先のローカルフォルダーを指定します。

- ***Remove-OrchJobMedia [-JobId] <string[]> [-Path <string[]>] [-Recurse] [-Depth <uint>] [-WhatIf] [-Confirm]***

必要な権限: OR.Monitoring,Write

ジョブから、ジョブ記録を削除します。先頭のパラメーターに、ジョブ記録を削除したいジョブ ID を指定します。ジョブ ID はワイルドカードを含むテキストをカンマ区切りで複数指定できます。ジョブ ID の補完入力、Get-OrchJobMedia で取得済みのものが候補として表示されます。

ビデオが添付されたジョブを取得する

- ***Get-OrchJobVideo***

現在のフォルダーで、ビデオが添付されたジョブをすべて取得します。取得したジョブのビデオを確認するには、このジョブをブラウザで開いてください。これには、*Open-OrchJob <ジョブ ID>*とします。

- ***Get-OrchJobVideo -Recurse***

現在のフォルダーとそのサブフォルダーで、ビデオが添付されたジョブをすべて取得します。ルートフォルダーで実行すると、このテナントでスクリーンショットが添付されたジョブをすべて取得します。

スクリーンショットが添付されたジョブを取得する

- ***Get-OrchJobMedia***

現在のフォルダーで、スクリーンショットが添付されたジョブをすべて取得します。

- ***Get-OrchJobMedia -Recurse***

現在のフォルダーとそのサブフォルダーで、スクリーンショットが添付されたジョブをすべて取得します。ルートフォルダーで実行すると、このテナントでスクリーンショットが添付されたジョブをすべて取得します。

- ***Get-OrchJobMedia -Skip 3 -First 5***

現在のフォルダーのジョブ記録について、先頭の 3 行をスキップして次の 5 行を取得します。

スクリーンショットをファイルに保存する

- ***Export-OrchJobMedia***

現在のフォルダーに保存されているジョブの画面写真をすべてファイルに保存します。

パラメーターを指定しない場合には、C:ドライブのカレントロケーションに保存されます。ファイル名は次のようになります。

fid<フォルダーID>_<プロセス名>_<ジョブ ID>_ScreenCapture.zip

なお、C:ドライブのカレントロケーションをファイルエクスプローラーで開くには、*ii c:*とします。

- ***Export-OrchJobMedia -Recurse***

現在のフォルダーと、そのサブフォルダーに保存されているジョブの画面写真をすべてファイルに保存します。ルートフォルダーで実行すると、このドライブ（テナント）に保存されているすべての画面写真をファイルに保存します。

- ***Export-OrchJobMedia -Destination c:¥temp***

現在のフォルダーに保存されているジョブの画面写真をすべてファイルに保存します。

スクリーンショットを削除する

- ***Remove-OrchJobMedia <ジョブ ID>***

現在のフォルダーから、指定のジョブ ID のジョブ記録を削除します。ジョブ ID は、ワイルドカードを含むテキストをカンマ区切りで複数指定できます。

- ***Remove-OrchJobMedia <ジョブ ID> -WhatIf***

現在のフォルダーから、指定のジョブ ID のジョブ記録を削除します。-WhatIf は、実際には削除せず、削除対象とするジョブ記録を確認することを指示します。

- ***Remove-OrchJobRecording <ジョブ ID> -Recurse***

現在のフォルダーと、そのすべてのサブフォルダーから、指定のジョブ ID のジョブ記録を削除します。

ログの操作

- ***Get-OrchLog [-Id] <long[]> [[-Level] <string>] [-State <string[]>] [-TimeStampAfter <datetime>] [-Skip <ulong>] [-First <ulong>] [-Path <string[]>]***

必要な権限: OR.Monitoring.Read

ロボットの実行ログを取得します。

ログを取得する

- ***Get-OrchLog <ジョブ ID>***

ジョブ ID（-Id パラメーターの値）は、Get-OrchJob で取得済みのジョブ ID で補完できます。ジョブ ID は、カンマで区切って複数指定できます。

- ***Get-OrchLog <ジョブ ID> -Level <ログレベル>***

指定したログレベル以上の行だけを出力します。ログレベルは補完入力できます。-Level の既定値は Info です。

- ***Get-OrchLog <ジョブ ID> -Skip 100 -First 20***

最初の 100 行をとばして、次の 20 行のみ出力します。-Level などのほかのパラメーターと併用できます。

- ***Get-OrchLog <ジョブ ID> -TimeStampAfter '2024/01/11 0:00'***

Get-OrchLog は 1 万行を超えるログは取得できません。ログが切れた場合は、その続きを-TimeStampAfter パラメーターを指定して取得できます。

- ***Get-OrchLog -State Faulted [Ctrl+Space]***

-State パラメーターは、ジョブ ID の補完入力で一覧表示するジョブを絞り込みます。上記の例では、失敗したジョブの ID のみを一覧表示します。なお、-State パラメーターは、Get-OrchLog の実行結果には影響を与えません。

監査ログの操作

監査ログはテナントエンティティです。そのため、本節のコマンドレットはカレントドライブ（テナント）の監査ログを操作します。（カレントロケーションは動作に影響しません）-Path パラメーターには、操作対象のドライブ（テナント）を指定してください。ドライブ名は補完入力できます。カンマで区切って複数を同時に指定できます。なお、ドライブ名にワイルドカードは使えません。-Path を指定しない場合には、カレントドライブの監査ログを操作します。

- ***Get-OrchAuditLog [[-Last] <string>] [[-Component] <string[]>] [[-UserName] <string[]>] [[-Action] <string[]>] [-ExecutionTimeAfter <datetime>] [-ExecutionTimeBefore <datetime>] [-ExpandEntity] [-Skip <ulong>] [-First <ulong>] [-Path <string[]>] [<CommonParameters>]***
- ***Get-OrchAuditLog [-Id <string[]>] [-ExpandEntity] [-Skip <ulong>] [-First <ulong>] [-Path <string[]>] [<CommonParameters>]***

必要な権限: OR.Audit.Read

監査ログを取得します。最初のパラメーターセットでは、さまざまなパラメーターで取得する監査ログの条件を指定できます。ふたつめのパラメーターセットでは、-Id パラメーターに取得する監査ログの Id を直接指定できます。この Id の補完リストに表示されるのは、最初のパラメーターセットで取得した（キャッシュ済みの）監査ログの Id のみです。このため、まず最初のパラメーターセットで複数のジョブを取得した後、詳細に確認したいジョブがあれば、その Id を -Id パラメーターに指定して再実行してください。

-ExpandEntity は、監査ログの詳細を展開します。

監査ログを取得する

- ***Get-OrchAuditLog***

監査ログをすべて表示します。監査ログはテナントエンティティなので、どのフォルダーで実行しても同じ結果が得られます。

- ***Get-OrchAuditLog Day***

直近の 24 時間以内に出力された監査ログを表示します。

- ***Get-OrchAuditLog -Component Asset -ExpandEntity***

アセットに関する監査ログを表示します。**-ExpandEntity** パラメーターは、監査ログに含まれるエンティティ行を展開することを指示します。

- ***Get-OrchAuditLog -Id <監査ログ ID> -ExpandEntity***

指定の ID の監査ログを表示します。監査ログ ID の補完候補には、先に実行した *Get-OrchAuditLog* によりキャッシュされた ID が表示されます。

このほか、**-UserName** や **-Action** などのパラメーターで、表示する監査ログをフィルターできます。

ロボットの操作

ロボットはテナントエンティティです。そのため、本節のコマンドレットはカレントドライブ（テナント）のロボットを操作します。（カレントロケーションは動作に影響しません）**-Path** パラメーターには、操作対象のドライブ（テナント）を指定してください。ドライブ名は補完入力できます。カンマで区切って複数を同時に指定できます。なお、ドライブ名にワイルドカードは使えません。**-Path** を指定しない場合には、カレントドライブのロボットを操作します。

- ***Get-OrchRobot [[-FullName] <string[]>] [-Path <string[]>]***

必要な権限: OR.Robots.Read

ロボットを取得します。

テナントのすべてのロボットを表示する

- ***Get-OrchRobot***

現在のドライブ（テナント）のロボットをすべて表示します。

- ***Get-OrchRobot -Path Orch1;,Orch2:***

指定したドライブ（テナント）のロボットをすべて表示します。**-Path** パラメーターでは、ドライブ名を補完入力できます。

指定した条件のロボットを表示する

- ***Get-OrchRobot <ロボット名>***

現在のドライブ（テナント）で、指定した名前のロボットをすべて表示します。ロボット名は補完入力できます。ワイルドカードを利用できます。カンマで区切って複数と同時に指定できます。

- ***Get-OrchRobot / ? {\$_.Type -eq "StudioPro"}***

Type が StudioPro となっているロボットをすべて表示します。

ユーザーとグループの操作

ユーザーとグループはテナントエンティティです。そのため、本節のコマンドレットはカレントドライブ（テナント）のユーザーとグループを操作します。（カレントロケーションは動作に影響しません）-Path パラメーターには、操作対象のドライブ（テナント）を指定してください。ドライブ名は補完入力できます。カンマで区切って複数と同時に指定できます。なお、ドライブ名にワイルドカードは使えません。-Path を指定しない場合には、カレントドライブのユーザーを操作します。

- ***Get-OrchUser [[-FullName] <string[]>] [[-UserName] <string[]>] [-Path <string[]>]***

必要な権限: OR.Users.Read

ユーザーを取得します。最初のパラメーターで、取得したいユーザーの FullName をカンマ区切りとワイルドカードで複数指定します。FullName を省略すると、すべてのユーザーを取得します。-UserName パラメーターで、UserName を指定することもできます。

- ***Remove-OrchUser [[-FullName] <string[]>] [[-UserName] <string[]>] [-Path <string[]>] [-WhatIf] [-Confirm]***

必要な権限: OR.Users.Write

ユーザーを削除します。ユーザーは-FullName もしくは-UserName で指定できます。

- ***Get-OrchCurrentUser [-Path <string[]>]***

必要な権限: OR.User.Read

指定のドライブに接続中のユーザーを表示します。非機密アプリ（ユーザースコープ）で接続したドライブに対してお使いください。機密アプリ（アプリスコープ）で接続したドライブに対しては動作しません。

- ***Update-OrchCurrentUserURPassword [-Path <string[]>] [-WhatIf] [-Confirm]***

必要な権限: OR.Users

現在のユーザーの Unattended Robot のパスワードを更新します。このコマンドレット実行直後に、対話形式でパスワード入力が必要です。

テナントのユーザーを表示する

- ***Get-OrchUser***

このドライブ（テナント）のすべてのユーザーを表示します。

- ***Get-OrchUser -Path Orch1;,Orch2:***

-Path パラメーターで指定したドライブ（テナント）のすべてのユーザーを表示します。

- ***Get-OrchUser a*,y****

このドライブ（テナント）の、a で始まるユーザーと y で始まるユーザーをすべて表示します。ユーザーの FullName で検索します。-FullName に指定する値は、[Tab]もしくは[Ctrl+Space]キーを押下して補完入力できます。

- ***Get-OrchUser -UserName a*,y****

このドライブ（テナント）の、a で始まるユーザーと y で始まるユーザーをすべて表示します。-UserName パラメーターで指定した場合には、ユーザーの UserName で検索します。-UserName に指定する値も補完入力できます。

- ***Get-OrchUser / ? { \$_.RolesList -contains 'Administrator' }***

Administrator ロールをもつユーザーのみを表示します。

テナントのユーザーを削除する

- ***Remove-OrchUser <ユーザー名>***

指定のユーザー名のユーザーを、このドライブ（テナント）から削除します。このユーザー名は FullName です。ユーザー名にはワイルドカードが使えます。カンマ区切りで複数を同時に指定することもできます。補完入力もできます。

- ***Remove-OrchUser -UserName <ユーザー名>***

指定のユーザー名のユーザーを、このドライブ（テナント）から削除します。-UserName パラメーター名を指定した場合には、UserName で検索します。ユーザー名にはワイルドカードが使えます。カンマ区切りで複数を同時に指定することもできます。補完入力もできます。

- ***Remove-OrchUser -Path Orch1: <ユーザー名>***

ドライブ名を指定することもできます。ドライブ名は、複数をカンマ区切りで指定で

きますが、ワイルドカードは使えません。

現在のユーザーを確認する

Get-OrchCurrentUser コマンドレットは、非機密アプリ（ユーザースコープ）で接続したドライブで利用できます。機密アプリ（アプリスコープ）で接続したドライブでは動作しないことに注意してください。

- ***Get-OrchCurrentUser***

現在のドライブ（テナント）に、UiPathOrch でログインしているユーザーを表示します。このユーザーは、ドライブごとに異なる可能性があることに注意してください。

- ***Get-OrchCurrentUser -Path Orch1;,Orch2:***

指定のドライブ（テナント）に、UiPathOrch でログインしているユーザーを表示します。-Path パラメーターには複数のドライブをカンマ区切りで指定できます。補完入力もできます。

現在のユーザーの Unattended Robot のパスワードを更新する

Update-OrchCurrentUserURPassword コマンドレットは、非機密アプリ（ユーザースコープ）で接続したドライブで利用できます。機密アプリ（アプリスコープ）で接続したドライブでは動作しないことに注意してください。そのため、ロボットアカウントの Unattended Robot パスワードを変更することもできません。変更できるのは、ユーザーアカウントの Unattended Robot パスワードのみです。

- ***Update-OrchCurrentUserURPassword***

パラメーターを一切指定せずに実行してください。対話形式で、パスワード入力を要求します。現在のユーザーの Unattended Robot が Windows にログインするときのパスワードが更新されます。

- ***Update-OrchCurrentUserURPassword -Path Orch1;,Orch2:***

-Path パラメーターで、対象のドライブ（テナント）を指定できます。複数のドライブを指定すると、それらに接続中の現在のユーザーの Unattended Robot のパスワードを一括して同じパスワードに更新します。-WhatIf パラメーターをつけると、各ドライブの現在のユーザーを確認できます。

ロールの操作

ロールはテナントエンティティです。そのため、本節のコマンドレットはカレントドライブ（テナント）のロールを操作します。（カレントロケーションは動作に影響しません） -Path パラメーターには、操作対象のドライブ（テナント）を指定してください。ドライブ

名は補完入力できます。カンマで区切って複数を同時に指定できます。なお、ドライブ名にワイルドカードは使えません。-Path を指定しない場合には、カレントドライブのロールを操作します。

- ***Get-OrchRole* *[-DisplayName]* <string[]> *[-Path]* <string[]> *[-ExpandPermission]***

必要な権限: OR.Users.Read

ロールを取得します。スイッチパラメーター-ExpandPermission は、ロールに含まれる権限を展開して表示することを指示します。

- ***Remove-OrchRole* *[-DisplayName]* <string[]> *[-Path]* <string[]> *[-WhatIf]* *[-Confirm]***

必要な権限: OR.Users.Write

ロールを削除します。

- ***Copy-OrchRole* *[-DisplayName]* <string[]> *[-Destination]* <string[]> *[-Path]* <string> *[-WhatIf]* *[-Confirm]***

必要な権限: コピー元テナントの OR.Users.Read

コピー先テナントの OR.Users.Write

テナント間でロールをコピーします。

ロールを表示する

- ***Get-OrchRole***

現在のドライブ（テナント）のロールをすべて表示します。

- ***Get-OrchRole* <ロール名>**

現在のドライブ（テナント）の、指定の名前のロールをすべて表示します。ロール名には、ワイルドカードを含む名前をカンマ区切りで複数指定できます。補完入力もできます。

- ***Get-OrchRole* <ロール名> *-ExpandPermission***

-ExpandPermission パラメーターを指定すると、このロールに含まれる権限の詳細が展開されて表示されます。

ロールを削除する

- ***Remove-OrchRole* <ロール名>**

指定の名前のロールをテナントから削除します。ワイルドカードを含む名前をカンマ区切りで複数指定できます。補完入力もできます。

- ***Remove-OrchRole -Path Orch1;,Orch2 a****

指定のドライブ（テナント）のすべてで、a で始まるロールをすべて削除します。

- ***Remove-OrchRole -Path Orch1;,Orch2 a* -WhatIf***

指定のドライブ（テナント）のすべてで、a で始まるロールをすべて削除します。 -WhatIf パラメーターは、実際には削除せず、削除対象とするロール名を確認することを指示します。-WhatIf の代わりに-Confirm を指定すると、ロールをひとつ削除するたびに確認メッセージを表示します。

テナント間でロールをコピーする

- ***Copy-OrchRole * Orch2:***

現在のドライブ（テナント）のすべてのロールを、Orch2:にコピーします。

- ***Copy-OrchRole * Orch2: -WhatIf***

現在のドライブ（テナント）のすべてのロールを、Orch2:にコピーします。-WhatIf パラメーターは、実際にはコピーせず、コピー対象とするロール名を確認することを指示します。-WhatIf の代わりに-Confirm を指定すると、ロールをひとつコピーするたびに確認メッセージを表示します。

- ***Copy-OrchRole -Path Orch2: a*.***

Orch2:のドライブ（テナント）の a ではじまるロールすべてを、カレントドライブのテナントにコピーします。

- ***Copy-OrchRole * Orch2;,Orch3:***

現在のドライブ（テナント）のすべてのロールを、Orch2:と Orch3:の両方にコピーします。

フォルダーユーザーの操作

本節のコマンドレットは、-FullName か-UserName のどちらか、もしくは両方を使って操作対象のユーザーを指定できます。-FullName の方が使いやすいと思いますが、-FullName では操作対象が一意とならない場合があることに注意してください。-FullName は位置指定パラメーターとなっているため、パラメーター名を指定する必要はありません。-UserName でユーザーを指定する場合は、-UserName のパラメーター名とあわせて指定してください。FullName と UserName のどちらにもワイルドカードが使えます。操作対象となるユーザーを確認するには、-WhatIf パラメーターが役に立ちます。ロールにもワイルドカードが使えます。

- ***Get-OrchFolderUser [[-FullName] <string[]>] [[-UserName] <string[]>] [-IncludeInherited] [-Path <string[]>] [-Recurse] [-Depth <uint>]***

必要な権限: OR.Folders.Read

フォルダーに割り当てられたユーザーを取得します。

- ***Add-OrchFolderUser*** *[[**-FullName**] <string[]>] [**-Roles**] <string[]> [**-UserName**] <string[]>] [**-Path** <string[]>] [**-WhatIf**] [**-Confirm**]*

必要な権限: OR.Folders

フォルダーにユーザーを割り当てます。

- ***Remove-OrchFolderUser*** *[[**-FullName**] <string[]>] [**-UserName**] <string[]>] [**-Path**] <string[]>] [**-Recurse**] [**-Depth** <uint>] [**-WhatIf**] [**-Confirm**]*

必要な権限: OR.Folders

フォルダーからユーザーを削除します。

- ***Add-OrchRoleToFolderUser*** *[[**-FullName**] <string[]>] [**-Roles**] <string[]> [**-UserName**] <string[]>] [**-Path** <string[]>] [**-Recurse**] [**-Depth** <uint>] [**-WhatIf**] [**-Confirm**]*

必要な権限: OR.Folders

フォルダーユーザーにロールを追加します。

- ***Remove-OrchRoleFromFolderUser*** *[[**-FullName**] <string[]>] [**-Roles**] <string[]> [**-UserName**] <string[]>] [**-Path** <string[]>] [**-Recurse**] [**-Depth** <uint>] [**-WhatIf**] [**-Confirm**]*

必要な権限: OR.Folders

フォルダーユーザーからロールを削除します。

フォルダーに割り当てられたユーザーを表示する

- ***Get-OrchFolderUser***

現在のフォルダーに割り当てられているユーザーをすべて表示します。なお、親フォルダーから継承したユーザーは表示しません。

- ***Get-OrchFolderUser -IncludeInherited***

親フォルダーから継承したユーザーも表示します。

- ***Get-OrchFolderUser -Recurse***

現在のフォルダーと、そのサブフォルダーに割り当てられているユーザーをすべて表示します。ルートフォルダーで実行すると、すべてのフォルダーのユーザーを表示します。

- ***Get-OrchFolderUser -IncludeInherited***

現在のフォルダーに割り当てられているユーザーをすべて表示します。親フォルダーから継承したユーザーも表示します。

- ***Get-OrchFolderUser *yoshi****

現在のフォルダーに割り当てられている、FullName に yoshi を含むユーザーをすべて表示します。ユーザー名はワイルドカードを含む名前をカンマ区切りで複数指定できます。補完入力もできます。UserName で検索したい場合には、-UserName パラメーターで指定します。

- ***Get-OrchFolderUser -Path <フォルダー名>***

指定のフォルダーに割り当てられたユーザーを表示します。ドライブ文字を含むパス（Orch1:¥shared など）を指定すると、別のプロバイダーのドライブから実行することもできます。フォルダー名にはワイルドカードを含む名前をカンマ区切りで複数指定できます。

フォルダーからユーザーを削除する

- ***Remove-OrchFolderUser ユーザー名***

現在のフォルダーに割り当てられているユーザーを、フォルダーから削除します。ユーザー名は、FullName で検索されます。ユーザー名には、ワイルドカードを含む名前をカンマ区切りで複数指定できます。

- ***Remove-OrchFolderUser -Recurse a****

FullName が a で始まるユーザーを、現在のフォルダーとそのサブフォルダーのすべてから削除します。

- ***Remove-OrchFolderUser -Recurse -UserName a* -WhatIf***

UserName が a で始まるユーザーを、現在のフォルダーとそのサブフォルダーのすべてから削除します。-WhatIf パラメーターは、実際には削除せず、削除対象となるユーザーを確認することを指示します。

- ***Remove-OrchFolderUser -Recurse -UserName a* -Confirm***

UserName が a で始まるユーザーを、現在のフォルダーとそのサブフォルダーのすべてから削除します。-Confirm パラメーターは、ユーザーをひとつ削除するたびに確認メッセージを表示します。

フォルダーにユーザーを割り当てる

- ***Add-OrchRoleToFolderUser <ユーザー名> <ロール名>***

現在のフォルダーにユーザーを割り当てます。ユーザー名は FullName で指定します。UserName で指定したい場合には -UserName パラメーターで指定します。ユーザー名

は、ワイルドカードを含む名前をカンマ区切りで複数指定できます。補完入力もできます。この補完入力は、フォルダーに割り当て済みのユーザーのみを表示します。少なくともひとつのロールを指定する必要があります。ロールも、ワイルドカードを含む名前をカンマ区切りで複数指定できます。補完入力もできます。

- ***Add-OrchRoleToFolderUser*** <ユーザー名> <ロール名> -WhatIf

上記と同様ですが、実際には追加せず、追加の対象となるユーザーを確認します。

フォルダーに割り当て済みのユーザーから、ロールを削除する

- ***Remove-OrchRoleFromFolderUser*** <ユーザー名> <ロール名>

ユーザー名とロール名は、それぞれワイルドカードを含む名前をカンマ区切りで複数指定できます。補完入力では、現在のフォルダーに割り当て済みのユーザーとロールのみが表示されます。

マシンの操作

マシンはテナントエンティティです。そのため、本節のコマンドレットはカレントドライブ（テナント）のマシンを操作します。（カレントロケーションは動作に影響しません） -Path パラメーターには、操作対象のドライブ（テナント）を指定してください。

- ***Get-OrchMachine*** *[-Name]* <string[]> *[-Path]* <string[]> *[-ExpandRobotUser]*

必要な権限: OR.Machines.Read

マシンを取得します。-ExpandRobotUser は、各マシンのアカウントとロボットのマッピングを表示します。

- ***Remove-OrchMachine*** *[-Name]* <string[]> *[-Path]* <string[]> *[-WhatIf]* *[-Confirm]*

必要な権限: OR.Machines.Write

マシンを削除します。

- ***Copy-OrchMachine*** *[-Name]* <string[]> *[-Destination]* <string[]> *[-Path]* <string> *[-WhatIf]* *[-Confirm]*

必要な権限: コピー元テナントの OR.Machines.Read

コピー先テナントの OR.Machines.Write

テナント間で、マシンをコピーします。

テナントのマシンを表示する

- ***Get-OrchMachine***
このドライブ（テナント）のマシンをすべて表示します。
- ***Get-OrchMachine <マシン名>***
このドライブ（テナント）の、指定した名前のマシンをすべて表示します。マシン名には、ワイルドカードを含む名前をカンマ区切りで複数指定できます。補完入力もできます。
- ***Get-OrchMachine -Path Orch1;Orch2***
指定のドライブ（テナント）のマシンをすべて表示します。
- ***Get-OrchMachine -ExpandRobotUser***
各マシンのアカウントとマシンのマッピングを表示します。

テナントのマシンを削除する

- ***Remove-OrchMachine <マシン名>***
このドライブ（テナント）の、指定した名前のマシンをすべて削除します。マシン名には、ワイルドカードを含む名前をカンマ区切りで複数指定できます。補完入力もできます。

テナント間で、マシンをコピーする

標準マシンを作成できない環境においては、このコマンドレットは標準マシンをコピーすることはできないことにご留意下さい。

- ***Copy-OrchMachine <マシン名> Orch2:***
現在のドライブの指定したマシンを、Orch2:ドライブにコピーします。マシン名にはワイルドカードを含む名前をカンマ区切りで複数指定できます。補完もできます。コピー先ドライブ(-Destination パラメーター)には、ドライブを複数指定できます。
- ***Copy-OrchMachine <マシン名> Orch2: -WhatIf***
実際にはコピーせず、コピー対象となるマシン名を確認します。-WhatIf の代わりに-Confirm を指定すると、マシンをひとつコピーするたびに確認メッセージを表示します。

フォルダーマシンの操作

- ***Get-OrchFolderMachine [[-Name] <string[]>] [-Path <string[]>] [-Recurse] [-Depth <uint>]***

必要な権限: OR.Folders.Read

フォルダーに割り当てられたマシンを取得します。

- ***Add-OrchFolderMachine [-Name] <string[]> [-Path <string[]>] [-Recurse] [-Depth <uint>] [-WhatIf] [-Confirm]***

必要な権限: OR.Folders.Write

マシンを、フォルダーに割り当てます。

- ***Remove-OrchFolderMachine [-Name] <string[]> [-Path <string[]>] [-Recurse] [-Depth <uint>] [-WhatIf] [-Confirm]***

必要な権限: OR.Folders.Write

フォルダーに割り当てられたマシンを、フォルダーから削除します。

フォルダーマシンを取得する

- ***Get-OrchFolderMachine***

現在のフォルダーに割り当てられているマシンをすべて表示します。

- ***Get-OrchFolderMachine -Recurse***

現在のフォルダーと、そのサブフォルダーすべてに割り当てられているマシンをすべて表示します。ルートフォルダーで実行すると、すべてのフォルダーに割り当てられているマシンを表示します。

- ***Get-OrchFolderMachine -Recurse <マシン名>***

指定のマシンが割り当てられているフォルダーを、現在のフォルダーとそのサブフォルダーから検索します。マシン名は、ワイルドカードを含む名前をカンマ区切りで複数指定できます。

マシンをフォルダーに割り当てる

- ***Add-OrchFolderMachine <マシン名>***

現在のフォルダーに、指定したマシンを割り当てます。マシン名は、ワイルドカードを含む名前をカンマ区切りで複数指定できます。

- ***Add-OrchFolderMachine <マシン名> -WhatIf***

-WhatIf パラメーターは、実際に追加はせず、追加対象となるマシンを確認します。

フォルダーマシンを削除する

- ***Remove-OrchFolderMachine* <マシン名>**

現在のフォルダーから、指定のマシンを削除します。マシン名は、ワイルドカードを含む名前をカンマ区切りで複数指定できます。

- ***Remove-OrchFolderMachine* <マシン名> -WhatIf**

-WhatIf パラメーターは、実際に削除はせず、削除対象となるマシンを確認します。

アセットの操作

- ***Get-OrchAsset* [[-Name] <string[]>] [-ValueType <string[]>] [-ExpandUserValue] [-Path <string[]>] [-Recurse] [-Depth <uint>]**

- ***Get-OrchAsset -ExportCsv* [[-Encoding] <Encoding>] [[-CsvPath] <string>] [-Path <string[]>] [-Recurse] [-Depth <uint>]**

- ***Get-OrchAsset -ExportCredentialCsv* [[-Encoding] <Encoding>] [[-CsvPath] <string>] [-Path <string[]>] [-Recurse] [-Depth <uint>]**

必要な権限: OR.Assets.Read

アセットを取得します。最初のパラメーターセットは、-Name と -ValueType で指定したアセットを取得します。スイッチパラメーター -ExpandUserValue は、ユーザー・マシンごとの値を展開して表示することを指示します。

ふたつめのパラメーターセットのパラメーター -ExportCsv は、テキスト・ブール・整数のアセットを CSV ファイルに出力します。

みつめめのパラメーターセットのパラメーター -ExportCredentialCsv は、クレデンシャルアセットを CSV ファイルに出力します。

このコマンドレットで出力した CSV ファイルは、そのまま *Set-OrchAsset* と *Set-OrchCredentialAsset* コマンドレットでインポートできます。

- ***Set-OrchAsset* [[-ValueType] <string>] [-Name] <string[]> [[-Value] <string>] [[-UserName] <string[]>] [[-MachineName] <string[]>] [-Description <string>] [-Path <string[]>] [-WhatIf] [-Confirm]**

- ***Set-OrchAsset -ExportTemplateCsv* [[-Encoding] <Encoding>] [[-TemplateCsvPath] <string>] [-WhatIf] [-Confirm]**

必要な権限: OR.Assets

最初のパラメーターセットは、アセットを作成・更新・削除します。存在しないアセット名を指定すると、アセットを新規作成します。既存のアセット名を指定すると、このアセットを更新します。このとき、-Value パラメーターに " (空文字列)を指定す

ると、このアセット値を削除します。

ふたつめのパラメーターセットのパラメーター-ExportTemplateCsv は、この *Set-OrchAsset* コマンドレットでインポートできる CSV ファイルのテンプレートを出力します。

- ***Remove-OrchAsset [-Name] <string[]> [[-ValueType] <string[]>] [-Path <string[]>] [-Recurse] [-Depth <uint>] [-WhatIf] [-Confirm]***

必要な権限: OR.Assets.Write

アセットを削除します。上述の *Set-OrchAsset* と *Set-OrchCredentialAsset* は、指定したユーザー・マシンごとのアセット値を削除できますが、*Remove-OrchAsset* はユーザー・マシンごとのアセット値の有無に関わらず、指定したアセット自体を削除します。

アセットとクレデンシャルアセットを表示する

- ***Get-OrchAsset***

現在のフォルダーのアセットをすべて表示します。なお、クレデンシャルアセットのパスワードは表示されません。これは Orchestrator Web API の仕様による制限です。

- ***Get-OrchAsset <アセット名>***

現在のフォルダーのアセットをすべて表示します。アセット名には、ワイルドカードを含む名前をカンマ区切りで複数指定できます。

- ***Get-OrchAsset <アセット名> -ExpandUserValue***

現在のフォルダーの指定のアセットのユーザーごと・マシンごとのアセット値を展開して表示します。グローバル値が設定されていれば、グローバル値も最初に出力します。

- ***Get-OrchAsset -Recurse***

現在のフォルダーと、そのすべてのサブフォルダーのアセットをすべて表示します。

- ***Get-OrchAsset -ValueType Text***

現在のフォルダーのテキストアセットをすべて表示します。上記のコマンドレットの直後に補完の操作をすると、テキストアセットの名前だけが一覧表示されます。

アセットを追加・更新する

- ***Set-OrchAsset Text <アセット名> アセット値***

現在のフォルダーに、Text 型で指定の名前のアセットを作成し、そのグローバル値に 'アセット値' を設定します。既に同名のアセットがあれば、そのグローバル値を更新します。アセット名には、ワイルドカードを含む名前をカンマ区切りで複数指定でき

ます。ワイルドカードを展開しても既存のアセットにひとつも合致しない場合には、新規アセット名として扱われます。アセット値を補完入力すると、現在の値が表示されます。

- ***Set-OrchAsset Bool* <アセット名> アセット値**

Bool 型と Integer 型のアセットについても、同様に動作します。

- ***Set-OrchAsset Text* <アセット名> <アセット値> <ユーザー名>**

ユーザー名を指定すると、ユーザーごとのアセット値を作成・更新できます。ユーザー名の補完は、対象のフォルダーに割り当てられたユーザー一覧のみを表示します。ただし、ユーザー名にワイルドカードを指定したとき、ユーザーはテナントのユーザーすべてを対象に検索・展開されることに注意してください。これは、CSV ファイルからのインポート時に高速に処理できるようにするためです。(フォルダーユーザーの取得は、Orchestrator において比較的高いコストが高い処理です)

- ***Set-OrchAsset Text* <アセット名> -UserName <ユーザー名> [Ctrl+Space]**

ユーザーごとのアセット値の現在の値を編集して更新するには、アセット値より先にユーザー名を指定してから補完入力の操作をしてください。現在の値が表示されます。

- ***Set-OrchAsset Text* <アセット名> <アセット値> <ユーザー名> <マシン名>**

ユーザー名とマシン名の両方を指定すると、ユーザーごと・マシンごとのアセット値を作成・更新できます。

- ***Set-OrchAsset Text* <アセット名> -UserName <ユーザー名> -MachineName <マシン名> [Ctrl+Space]**

ユーザーごと・マシンごとのアセット値の現在の値を編集して更新するには、アセット値より先にユーザー名とマシン名を指定してから補完入力の操作をしてください。

- ***Set-OrchAsset Text* <アセット名> -Description <説明>**

現在のフォルダーの指定のアセットに、説明を追加/更新します。なお、このコマンドレットは -Description パラメーターに " を指定しても、アセットの説明を削除することはありません。これは、インポートする CSV ファイルの Description 列に空文字列を指定しても、既存の Description を削除しないようにするための動作です。

アセットを削除する

- ***Remove-OrchAsset* <アセット名>**

指定のクレデンシャルアセットを完全に削除するには、*Remove-OrchAsset* コマンドレットを使います。アセット名にはワイルドカードを含む名前をカンマ区切りで複数指定できます。-Recurse パラメーターを利用できます。

- ***Set-OrchAsset Text* <アセット名> "**

アセット値に " を指定すると、このアセットの値を削除します。ユーザーごと・マシン

ンごとのアセット値がひとつでも含まれているアセットの場合には、グローバル値だけが削除され、既存のアセット自体は削除されません。ユーザーごとの値を含まないアセットに対して上記を実行した場合は、このアセットは完全に削除されます。

- ***Set-OrchAsset Bool <アセット名> "***

Bool 型と Integer 型のアセットについても、アセット値に " を指定することで削除できます。上の例では -ValueType パラメーターに Bool を指定しています。なお、このパラメーターの指定が必須となるのは、新規アセット作成時だけです。既存のアセットを更新する場合は、-ValueType パラメーターは指定しなくても動作します。ただし、-ValueType は位置指定パラメーターなので、このパラメーターを省略したときは後続のパラメーターを名前付きで指定する必要があります。

- ***Set-OrchAsset Text <アセット名> "<ユーザー名>***

ユーザー名を指定すると、そのユーザー名のユーザーごとのアセット値を削除します。この場合、グローバル値と、マシン名をもつアセット値は削除されません。ユーザー名には、ワイルドカードを含む名前をカンマ区切りで複数指定できます。補完もできます。このアセットに含まれるユーザーごとのアセットをすべて削除するには、ユーザー名に * を指定します。

- ***Set-OrchAsset Text <アセット名> "<ユーザー名> <マシン名>***

ユーザー名とマシン名の両方を指定すると、合致するユーザーごと・マシンごとのアセット値を削除します。ユーザー名とマシン名には、ワイルドカードを含む名前をカンマ区切りで複数指定できます。補完もできます。このアセットに含まれる、ユーザー名・マシン名の両方をもつアセット値をすべて削除するには、ユーザー名とマシン名の両方に * を指定します。

アセットを CSV ファイルにエクスポートする

CSV ファイルにエクスポートするときは、-ExpandUserValue パラメーターを指定する必要はありません。自動で、ユーザーごと・マシンごとのアセット値も出力されます。本節で出力した CSV ファイルは、そのまま *Set-OrchAsset* コマンドレットでインポートできます。この方法は後述します。

- ***Get-OrchAsset -ExportCsv -Encoding shift_jis***

現在のフォルダーにある Text/Bool/Integer のアセットを、CSV ファイルに出力します。CSV ファイルのパスを指定しない場合には、C:ドライブのカレントロケーションに既定のファイル名 *ExportedAssets.csv* で出力されます。-Encoding パラメーターで CSV ファイルのエンコーディングを指定できます。日本語の環境においては、shift_jis で出力しておく Excel で文字化けすることなく開くことができ便利です。

- ***Get-OrchAsset -ExportCsv -Encoding shift_jis assets.csv***

指定したファイル名で、C:ドライブのカレントロケーションに出力します。なお、C:のカレントロケーションに移動するには `cd c:` とします。C:のカレントロケーションをファイルエクスプローラーで開くには、`ii c:` とします。また、C:のカレントロケーションにあるファイル `assets.csv` を開くには、`c:asset.csv` とします。`c:a[Ctrl+Space]` でファイル名を補完入力することもできます。

- ***Get-OrchAsset -ExportCsv -Encoding shift_jis data/assets.csv***

C:ドライブのカレントロケーションからの相対パスを指定する例です。C:ドライブ (FileSystem プロバイダ) のフォルダーの区切り文字にも、/もしくは¥の両方が使えます。

- ***Get-OrchAsset -ExportCsv -Encoding shift_jis c:¥temp***

C:ドライブのフォルダーの絶対パスを指定する例です。この場合には、指定のフォルダーに既定のファイル名 `ExportedAssets.csv` が出力されます。

- ***Get-OrchAsset -ExportCsv -Encoding shift_jis c:¥temp***

C:ドライブのファイルの絶対パスを指定する例です。この場合には、指定のファイル名で出力されます。

アセットを CSV ファイルからインポートする

Set-OrchAsset は、CSV ファイルに Name 列 (アセット名) が同じ行が複数あるとき、それらの行をひとつにまとめて 1 回の API 呼び出しにします。そのため、多くのユーザーごとの値を含む CSV ファイルも高速に処理します。CSV ファイルの Value 列が空になっているアセットは削除されます。

- ***Import-Csv C:ExportedAssets.csv -Encoding shift_jis / Set-OrchAsset -WhatIf***

CSV ファイルの各行にある列名をパラメーターとして、*Set-OrchAsset* を実行します。この CSV ファイルは、*Get-OrchAsset* に `-ExportCsv` を指定することで作成できます。インポート先のフォルダーは、CSV ファイルの Path 列で指定されていることに注意してください。`shift_jis` は、`sjis` としても同じです。

- ***Import-Csv C:ExportedAssets.csv -Encoding shift_jis / Set-OrchAsset -Path . -WhatIf***

CSV ファイルに記載されたすべてのアセットを現在のフォルダーにインポートするには、コマンドレットの `-Path` パラメーターでインポート先のフォルダーを指定してください。CSV ファイルの Path 列は、`-Path` パラメーターで指定されたフォルダーで上書きされます。上記の例では、CSV に記載されたアセットをすべて現在のフォルダーにインポートします (`-Path` パラメーターで指定されたフォルダーからの相対位置とは解釈されないことに注意してください)。

- ***Import-Csv C:ExportedAssets.csv -Encoding shift_jis / Set-OrchAsset -WhatIf***

もし、アセットをエクスポートしたドライブとは違うドライブ (テナント) にインポ

ートしたい場合には、CSV ファイルの Path 列を修正し、ドライブ文字をインポート先のドライブ文字に置換してください。あるいは、Path 列にあるドライブ文字を削除してルートフォルダーから始まる形式にして、上記のコマンドをインポート先のドライブで実行してください。

クレデンシャルアセットの操作

- ***Set-OrchCredentialAsset [-Name] <string[]> [[-UserName] <string[]>] [[-MachineName] <string[]>] [[-CredentialUsername] <string>] [[-CredentialPassword] <string>] [-Description <string>] [-CredentialStore <string>] [-Path <string[]>] [-WhatIf] [-Confirm]***
- ***Set-OrchCredentialAsset -ExportTemplateCsv [[-Encoding] <Encoding>] [[-TemplateCsvPath] <string>] [-WhatIf] [-Confirm]***

必要な権限: OR.Assets

最初のパラメーターセットは、クレデンシャルアセットを作成・更新・削除します。-Name パラメーターに存在しないアセット名を指定すると、アセットを新規作成します。-Name パラメーターに既存のアセット名を指定すると、このアセットを更新します。このとき、-CredentialPassword パラメーターに " (空文字列) を指定すると、このアセット値を削除します。

ふたつめのパラメーターセットのパラメーター-ExportTemplateCsv は、この *Set-OrchCredentialAsset* コマンドレットでインポートできる CSV ファイルのテンプレートを出力します。

クレデンシャルアセットを表示する

クレデンシャルアセットの表示は、*Get-OrchAsset* コマンドレットで行えます。前節を参照してください。なお Orchestrator API の制限により、パスワードは表示できません。

クレデンシャルアセットを追加・更新する

- ***Set-OrchCredentialAsset <クレデンシャルアセット名>***

現在のフォルダーにクレデンシャルアセットを作成します。対話形式でクレデンシャルのユーザー名とパスワードを要求するので、それらを PowerShell コンソールから入力してください。それらが、指定したアセット名のグローバル値に設定されます。既に同名のクレデンシャルアセットがあれば、そのグローバル値を更新します。アセット名にワイルドカード文字が含まれるとき、それを展開しても既存のアセットに合致しない場合には、新規にクレデンシャルアセットを作成します。

- ***Set-OrchCredentialAsset* <クレデンシャルアセット名> <ユーザー名>**
ユーザー名 (-UserName パラメーター) を指定した場合は、そのユーザー用のクレデンシャルユーザーとクレデンシャルパスワードを作成・更新します。ユーザー名の補完は、対象のフォルダーに割り当てられたユーザー一覧を表示します。ただし、ユーザー名にワイルドカードを指定したとき、ユーザーはテナントのユーザーすべてを対象に検索・展開されることに注意してください。これは、CSV ファイルからのインポート時に高速に処理できるようにするためです。(フォルダーユーザーの取得は、Orchestrator において比較的高いコストが高い処理です)
- ***Set-OrchCredentialAsset Text* <アセット名> <アセット値> <ユーザー名> <マシン名>**
ユーザー名とマシン名の両方を指定すると、ユーザーごと・マシンごとのアセット値を作成・更新できます。対話形式でクレデンシャルのユーザー名とパスワードを要求するので、それらを PowerShell コンソールから入力してください。
- ***Set-OrchAsset Text* <アセット名> -CredentialUsername <クレデンシャルユーザー名> -CredentialPassword <クレデンシャルパスワード>**
クレデンシャルのユーザー名とパスワードをパラメーターで指定することもできます。この場合は、対話形式での入力不要です。実際には、このパラメーターはインポートする CSV の列名として利用することが多いでしょう。

クレデンシャルアセットを削除する

- ***Remove-OrchAsset* <アセット名>**
指定のクレデンシャルアセットを完全に削除するには、Remove-OrchAsset コマンドレットを使います。アセット名にはワイルドカードを含む名前をカンマ区切りで複数指定できます。-Recurse パラメーターを利用できます。
- ***Set-OrchCredentialAsset* <アセット名> -CredentialPassword "**
-CredentialPassword パラメーターに " を指定すると、このアセットの値を削除します。ユーザーごと・マシンごとのアセット値がひとつでも含まれているクレデンシャルアセットに対して上記を実行したときは、グローバル値だけが削除され、このアセット自体は削除されません。ユーザーごとの値を含まないクレデンシャルアセットに対して上記を実行した場合には、このアセットは完全に削除されます。
- ***Set-OrchCredentialAsset* <アセット名> "<ユーザー名>**
ユーザー名を指定すると、そのユーザー名のユーザーごとのアセット値を削除します。この場合、グローバル値と、マシン名をもつアセット値は削除されません。ユーザー名には、ワイルドカードを含む名前をカンマ区切りで複数指定できます。補完もできます。このアセットに含まれるユーザーごとのアセットをすべて削除するには、ユーザー名に * を指定します。
- ***Set-OrchCredentialAsset Text* <アセット名> "<ユーザー名> <マシン名>**

ユーザー名とマシン名の両方を指定すると、合致するユーザーごと・マシンごとのアセット値を削除します。ユーザー名とマシン名には、ワイルドカードを含む名前をカンマ区切りで複数指定できます。補完もできます。このアセットに含まれる、ユーザー名・マシン名の両方をもつアセット値をすべて削除するには、ユーザー名とマシン名の両方に * を指定します。

クレデンシャルアセットを削除する

- ***Remove-OrchAsset <アセット名>***

指定のクレデンシャルアセットを完全に削除するには、Remove-OrchAsset コマンドレットを使います。アセット名にはワイルドカードを含む名前をカンマ区切りで複数指定できます。-Recurse パラメーターを利用できます。

クレデンシャルアセットを CSV ファイルにエクスポートする

本節で出力した CSV ファイルは、そのまま Set-OrchCredentialAsset コマンドレットでインポートできます。

- ***Get-OrchAsset -ExportCredentialCsv -Encoding shift_jis***

現在のフォルダーにある Credential のアセットを、CSV ファイルに出力します。CSV ファイルのパスとエンコーディングの指定方法は、-ExportCsv を指定したときと同じなので、前節を参考にしてください。

クレデンシャルアセットを CSV ファイルからインポートする

Set-OrchCredentialAsset は、CSV ファイルに Name 列（アセット名）が同じ行が複数あるとき、それらの行をひとつにまとめて 1 回の API 呼び出しにします。そのため、多くのユーザーごとの値を含む CSV ファイルも高速に処理します。CSV ファイルの CredentialUsername 列と CredentialPassword 列の両方が空になっているクレデンシャルアセットは削除されます。なお（CSV ファイルを使わず）コマンドレットパラメーターでユーザーごとのクレデンシャルを削除するときは、-CredentialPassword "を指定するだけで当該のクレデンシャルが削除されます（-CredentialUsername "を指定する必要はありません）。

- ***Import-Csv C:\ExportedCredentialAssets.csv -Encoding sjis / Set-OrchCredentialAsset -WhatIf***

CSV ファイルの各行にある列名をパラメーターとして、Set-OrchCredentialAsset を実行します。この CSV ファイルは、Get-OrchAsset に -ExportCredentialCsv を指定することで作成できます。インポート先のフォルダーは、CSV ファイルの Path 列で指定

されていることに注意してください。

- ***Import-Csv C:ExportedAssets.csv -Encoding sjis / Set-OrchAsset -Path . -WhatIf***

CSV ファイルに記載されたすべてのアセットを現在のフォルダーにインポートするには、コマンドレットの `-Path` パラメーターでインポート先のフォルダーを指定してください。CSV ファイルの `Path` 列は、`-Path` パラメーターで指定されたフォルダーで上書きされます。上記の例では、CSV に記載されたクレデンシャルアセットをすべて現在のフォルダーにインポートします (`-Path` パラメーターで指定されたフォルダーからの相対位置とは解釈されないことに注意してください)。

- ***Import-Csv C:ExportedCredentialAssets.csv -Encoding sjis / Set-OrchCredentialAsset -WhatIf***

もし、アセットをエクスポートしたドライブとは違うドライブ (テナント) にインポートしたい場合には、CSV ファイルの `Path` 列を修正し、ドライブ文字をインポート先のドライブ文字に置換してください。あるいは、`Path` 列のパスにフォルダー名からドライブ文字を削除してルートフォルダーから始まる形式にして、上記のコマンドをインポート先のドライブで実行してください。

資格情報ストアの操作

資格情報ストアはテナントエンティティです。そのため、本節のコマンドレットはカレントドライブ (テナント) の資格情報ストアを操作します。(カレントロケーションは動作に影響しません) `-Path` パラメーターには、操作対象のドライブ (テナント) を指定してください。

- ***Get-OrchCredentialStore [-Name] <string[]> [-Path <string[]>]***

必要な権限: OR.Settings.Read

資格情報ストアを取得します。

- ***Remove-OrchCredentialStore [-Name] <string[]> [-Path <string[]>] [-WhatIf] [-Confirm]***

必要な権限: OR.Settings.Write

資格情報ストアを削除します。

資格情報ストアを取得する

- ***Get-OrchCredentialStore***

現在のドライブ (テナント) の資格情報ストアをすべて表示します。

- ***Get-OrchCredentialStore <資格情報ストア名>***

指定の名前の資格情報ストアをすべて表示します。資格情報ストア名は、ワイルドカ

ードを含む名前をカンマ区切りで複数指定できます。補完もできます。

- ***Get-OrchCredentialStore -Path Orch1;,Orch2: <資格情報ストア名>***
-Path パラメーターに、対象のテナント（ドライブ）をカンマ区切りで複数指定できます。補完もできます。

資格情報ストアを削除する

- ***Remove-OrchCredentialStore <資格情報ストア名>***
現在のドライブ（テナント）から、指定の名前の資格情報ストアを削除します。
- ***Remove-OrchCredentialStore -Path Orch1;,Orch2 <資格情報ストア名>***
指定のドライブ（テナント）から、指定の名前の資格情報ストアを削除します。
- ***Remove-OrchCredentialStore -Path Orch1;,Orch2 <資格情報ストア名> -WhatIf***
-WhatIf パラメーターは、実際には削除せず、削除対象となる資格情報ストアを表示します。

キューの操作

- ***Get-OrchQueue [[-Name] <string[]>] [-Path <string[]>] [-Recurse] [-Depth <uint>]***
必要な権限: OR.Queues.Read
キューを取得します。
- ***Remove-OrchQueue [[-Name] <string[]>] [-Path <string[]>] [-WhatIf] [-Confirm]***
必要な権限: OR.Queues.Write
キューを削除します。
- ***Get-OrchQueueItem [-Name] <string[]> [-Skip <ulong>] [-First <ulong>] [-Path <string[]>]***
必要な権限: OR.Queues.Read
キューアイテムを取得します。
- ***Import-OrchQueueItem [-Name] <string[]> [-CsvPath] <string[]> [[-Encoding] <Encoding>] [[-CommitType] <string>] [-Path <string[]>] [-WhatIf] [-Confirm]***
必要な権限: OR.Queues.Write
CSV ファイルをアップロードして、キューアイテムを一括登録します。-CsvPath パラメーターには、ファイル名とフォルダーのいずれか（もしくは両方）をワイルドカードで指定できます。なお、同じファイルを複数回指定した場合でも、そのファイルは一度だけアップロードされます。

Upload a CSV file and bulk register queue items. The `-CsvPath` parameter allows you to specify either the file name, the folder, or both using wildcards. Note that even if the same file is specified multiple times, it will be uploaded only once.

キューを表示する

- ***Get-OrchQueue***
現在のフォルダーのキューをすべて表示します。
- ***Get-OrchQueue <キュー名>***
現在のフォルダーの指定の名前のキューをすべて表示します。キュー名には、ワイルドカードを含む名前をカンマ区切りで複数指定できます。補完もできます。
- ***Get-OrchQueue -Recurse***
現在のフォルダーと、そのサブフォルダーのキューをすべて表示します。ルートフォルダーで実行すると、このドライブ（テナント）のキューをすべて表示します。
- ***Get-OrchQueue -Recurse *cue****
現在のフォルダーとそのサブフォルダーで、名前に `cue` を含むキューをすべて表示します。ルートフォルダーで実行すると、このドライブ（テナント）のキューを検索できます。

キューを削除する

- ***Remove-OrchQueue <キュー名>***
現在のフォルダーの指定の名前のキューをすべて削除します。キュー名には、ワイルドカードを含む名前をカンマ区切りで複数指定できます。補完もできます。
- ***Remove-OrchQueue -Recurse <キュー名>***
現在のフォルダーとそのサブフォルダーで、指定の名前のキューをすべて削除します。
- ***Remove-OrchQueue <キュー名> -WhatIf***
`-WhatIf` パラメーターは、実際には削除せず、削除対象とするキュー名を確認します。

キューアイテムを取得する

- ***Get-OrchQueueItem <キュー名>***
現在のフォルダーの、指定のキューに含まれるアイテムを取得します。アイテムは、Id の昇順に表示されます。
- ***Get-OrchQueueItem <キュー名> / ?{\$_.Status -eq "New"}***
指定のキューに含まれるアイテムのうち、ステータスが `New` のものだけを取得します。

- ***Get-OrchQueueItem <キュー名> -Skip 5 -First 10***

指定のキューに含まれるアイテムのうち、先頭の 5 個をスキップして、その次の 10 個を取得します。

CSV ファイルをアップロードして、キューアイテムを一括登録する

- ***Import-OrchQueueItem <queue names> c:\temp\items.csv***

指定のキューに、指定の CSV ファイルをアップロードします。

- ***Import-OrchQueueItem <queue names> c:\temp\test*.csv shift_jis***

ワイルドカードを使って複数の CSV ファイルをまとめて指定できます。また、-Encoding パラメーターで CSV ファイルのエンコーディングを指定できます。

(Orchestrator web 画面では、UTF8 の CSV ファイルのみアップロードできます。)

- ***Import-OrchQueueItem <queue names> c:\temp***

-CsvPath パラメーターにフォルダーのパスを指定した場合には、そのフォルダーに含まれる拡張子が.csv のファイルをすべてアップロードします。

- ***Import-OrchQueueItem <queue names> c:\temp\items.csv shift_jis AllOrNothing***

-CommitType パラメーターで、登録時のストラテジーを指定できます。

AllOrNothing、StopOnFirstFailure、ProcessAllIndependently のいずれかを指定できます。-CommitType パラメーターを指定しない場合、ProcessAllIndependently が既定値として使われます。

トリガーの操作

本節のコマンドレットは、タイムトリガーとキュートリガーを操作します。イベントトリガーと API トリガーはサポートしないことに注意してください。

- ***Get-OrchTrigger [-Name] <string[]> [-Path <string[]>] [-Recurse] [-Depth <uint>]***

必要な権限: OR.Jobs.Read

トリガーを取得します。

- ***Remove-OrchTrigger [-Name] <string[]> [-Path <string[]>] [-Recurse] [-Depth <uint>] [-WhatIf] [-Confirm]***

必要な権限: OR.Jobs.Write

トリガーを削除します。

- ***Enable-OrchTrigger [-Name] <string[]> [-Path <string[]>] [-Recurse] [-Depth <uint>] [-WhatIf] [-Confirm]***

必要な権限: OR.Jobs.Write

トリガーを有効にします。

- ***Disable-OrchTrigger [-Name] <string[]> [-Path <string[]>] [-Recurse] [-Depth <uint>] [-WhatIf] [-Confirm]***

必要な権限: OR.Jobs.Write

トリガーを無効にします。

トリガーを表示する

- ***Get-OrchTrigger***

現在のフォルダーのすべてのトリガーを表示します。

- ***Get-OrchTrigger <トリガー名>***

現在のフォルダーの、指定の名前のトリガーを表示します。トリガー名には、ワイルドカードを含む名前をカンマ区切りで複数指定できます。補完もできます。

- ***Get-OrchTrigger -Recurse***

現在のフォルダーとそのサブフォルダーにある、すべてのトリガーを表示します。ルートフォルダーで実行すると、このテナントのすべてのトリガーを表示します。

- ***Get-OrchTrigger -Recurse / ? {\$_.Enabled -eq \$true}***

有効になっているトリガーをすべて表示します。

- ***Get-OrchTrigger -Recurse / select Path,Name,StartProcessCron***

各トリガーの Cron 式を表示します。

- ***Get-OrchTrigger / ? QueueDefinitionId -eq 66331 / select PackageName***

QueueDefinitionId が 66331 となっているキュートリガーのパッケージ名を表示します。なお、この簡潔化された条件式の構文は、PowerShell 3.0 で導入されました。

https://learn.microsoft.com/powershell/module/microsoft.powershell.core/about/about_simplified_syntax

トリガーを削除する

- ***Remove-OrchTrigger <トリガー名>***

現在のフォルダーの、指定の名前のトリガーを削除します。トリガー名には、ワイルドカードを含む名前をカンマ区切りで複数指定できます。補完もできます。トリガー名は省略できません。*を指定すると、現在のフォルダーのすべてのトリガーを削除します。

- ***Remove-OrchTrigger <トリガー名> -Confirm***

現在のフォルダーの、指定の名前のトリガーを削除します。トリガーをひとつ削除す

るたびに確認メッセージを表示します。

トリガーを有効にする

- ***Enable-OrchTrigger <トリガー名>***

現在のフォルダーの、指定の名前のトリガーを有効します。トリガー名には、ワイルドカードを含む名前をカンマ区切りで複数指定できます。補完もできます。トリガー名は省略できません。*を指定すると、現在のフォルダーのすべてのトリガーを有効にします。

- ***Enable-OrchTrigger <トリガー名> -WhatIf***

現在のフォルダーの、指定の名前のトリガーを有効します。-WhatIf は、実際には有効にせず、有効とする対象のトリガーを確認します。

トリガーを無効にする

- ***Disable-OrchTrigger <トリガー名>***

現在のフォルダーの、指定の名前のトリガーを無効にします。トリガー名には、ワイルドカードを含む名前をカンマ区切りで複数指定できます。補完もできます。トリガー名は省略できません。*を指定すると、現在のフォルダーのすべてのトリガーを無効にします。

- ***Disable-OrchTrigger -Recurse <トリガー名>***

現在のフォルダーとそのサブフォルダーの、指定の名前のトリガーを無効にします。ルートフォルダーで実行すると、このテナントで指定した名前のトリガーをすべて無効にします。

テストの操作

- ***Get-OrchTestCase [[-Name] <string[]>] [-Path <string[]>] [-Recurse] [-Depth <uint>]***

必要な権限: OR.TestSets.Read

テストケースを取得します。

- ***Get-OrchTestSet [[-Name] <string[]>] [-Path <string[]>] [-Recurse] [-Depth <uint>]***

必要な権限: OR.TestSets.Read

テストセットを取得します。

- ***Get-OrchTestExecution* *[-Name] <string[]>* *[-Path <string[]>* *[-Recurse] [-Depth <uint>]***

必要な権限: OR.TestSetExecutions.Read

テスト実行を取得します。

- ***Start-OrchTestSet* *[-Name] <string[]>* *[-Path <string[]>* *[-Recurse] [-Depth <uint>] [-WhatIf] [-Confirm]***

必要な権限: OR.TestSetExecutions.Write

テストセットを実行します。

- ***Stop-OrchTestExecution* *[-Id] <long[]>* *[-Path <string[]>* *[-WhatIf] [-Confirm]***

必要な権限: OR.TestSetExecutions.Write

テストセット実行を停止します。

テストケースを表示する

- ***Get-OrchTestCase***

現在のフォルダーのテストケースをすべて表示します。

- ***Get-OrchTestCase -Recurse***

現在のフォルダーとそのサブフォルダーのテストケースをすべて表示します。

- ***Get-OrchTestCase -Recurse -Path Orch1:¥***

Orch1:ドライブのテストケースをすべて表示します。

- ***Get-OrchTestCase* <テストケース名>**

現在のフォルダーの指定した名前のテストケースをすべて表示します。テストケース名は、ワイルドカードを含む名前をカンマ区切りで複数指定できます。補完もできます。

テストセットを表示する

- ***Get-OrchTestSet***

現在のフォルダーのテストセットをすべて表示します。

- ***Get-OrchTestSet -Recurse***

現在のフォルダーとそのサブフォルダーのテストセットをすべて表示します。

- ***Get-OrchTestSet -Recurse -Path Orch1:¥***

Orch1:ドライブのテストセットをすべて表示します。

- ***Get-OrchTestSet* <テストセット名>**

現在のフォルダーの指定した名前のテストセットをすべて表示します。テストセット名は、ワイルドカードを含む名前をカンマ区切りで複数指定できます。補完もできます。

テストセット実行を取得する

- ***Get-OrchTestSetExecution***
現在のフォルダーのテストセット実行をすべて表示します。
- ***Get-OrchTestExecution -Recurse***
現在のフォルダーとそのサブフォルダーのテストセット実行をすべて表示します。
- ***Get-OrchTestExecution -Recurse / ? Status -eq Failed***
- 現在のフォルダーとそのサブフォルダーで、失敗したテストセット実行をすべて表示します。
- ***Get-OrchTestExecution -Recurse -Path Orch1:¥***
Orch1:ドライブのテストセット実行をすべて表示します。
- ***Get-OrchTestExecution* <テストセット実行名>**
現在のフォルダーの指定した名前のテスト実行をすべて表示します。テスト実行名は、ワイルドカードを含む名前をカンマ区切りで複数指定できます。補完もできます。

テストセットを実行する

- ***Start-OrchTestSet* <テストセット名>**
現在のフォルダーで、指定のテストセットを実行します。テストセット名には、ワイルドカードを含む名前をカンマ区切りで複数指定できます。
- ***Start-OrchTestSet *-WhatIf***
現在のフォルダーにあるすべてのテストセットを実行します。-WhatIf パラメーターは、実際には実行せず、実行の対象となるテストセットを表示することを指示します。
- ***Start-OrchTestSet *-Confirm***
現在のフォルダーにあるすべてのテストセットを実行します。-Confirm パラメーターは、実行するかどうかをテストセットごとに確認します。
- ***Start-OrchTestSet *-Recurse -Verbose***
現在のフォルダーと、そのサブフォルダーにあるすべてのテストセットを実行します。-Verbose パラメーターは、実行するテストセットをコンソールに表示します。

テストセット実行を停止する

- ***Stop-OrchTestExecution* <テストセット実行Id>**

現在のフォルダーで、指定の Id のテストセット実行を停止します。この Id にはワイルドカードは指定できませんが、カンマ区切りで複数を同時に指定できます。補完入力もできます。

アラートの操作

アラートはテナントエンティティです。そのため、本節のコマンドレットはカレントドライブ（テナント）のアラートを操作します。（カレントロケーションは動作に影響しません）-Path パラメーターには、操作対象のドライブ（テナント）を指定してください。

- ***Get-OrchAlert* *[-Last]* <string> *[-Severity]* <string> *[-Component]* <string[]> *[-CreationTimeAfter]* <datetime> *[-CreationTimeBefore]* <datetime> *[-Skip]* <ulong> *[-First]* <ulong> *[-Path]* <string[]>**

必要な権限: OR.Monitoring.Read

アラートを取得します。

アラートを表示する

- ***Get-OrchAlert***

アラートをすべて表示します。

- ***Get-OrchAlert -Last Day***

過去24時間のアラートをすべて表示します。このほか、複数のパラメーターを同時に指定して、表示するアラートを絞り込めます。

その他の操作

- ***Clear-OrchCache* *[-Path]* <string[]>**

必要な権限: なし

UiPathOrch のメモリ内キャッシュをクリアします。Orchestrator Web や外部のツールで行った Orchestrator 上のエンティティの更新（フォルダーの作成や削除など）を UiPathOrch コンソールに反映するには、このコマンドレットでメモリ内キャッシュをクリアしてください。-Path パラメーターには、キャッシュをクリアしたいドライブを指定します。-Path パラメーターを指定しない場合は、カレントドライブのキャッシュをクリアします。このとき、カレントドライブが Orch ドライブでない場合には、すべての Orch ドライブのキャッシュをクリアします。

- ***Edit-OrchConfig* *[-Editor]* <EditConfigCommand+EditorType>**

必要な権限: なし

UiPathOrch の設定ファイルを開きます。-Editor には、Default もしくは Notepad を指定できます。指定しない場合、.json の拡張子に関連付けられた既定のテキストエディタで設定ファイルを開きます。設定ファイルのパスは、
%UserProfile%\Documents\PowerShell\Modules\UiPathOrch\UiPathOrchConfig.json です。